

Distributed In-Memory Cache with Strong Consistency Guarantees

(Rozproszona pamięć podręczna
z gwarancjami silnej spójności)

Dominik Szczepaniak

Praca inżynierska

Promotor: dr Piotr Witkowski

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

29 stycznia 2026

Abstract

This thesis presents the design and implementation of a distributed key-value storage system. It is designed to manage data across multiple servers with focus on data being truthful after each operation. Many modern systems like Redis, Apache Cassandra or Amazon DynamoDB focus on system being unavailable for as little time as possible. This often comes at the cost of temporary data inconsistency. This project ensures that all users see the exact same, up to date data regardless of the server that they connected to even in the event of network failures.

The system is built using an architecture that is separated into layers. This design allows to detect server failures and automatically reorganize itself without compromising any data integrity. The system is supported with a specialized client library that optimizes communication and ensures that requests are routed as efficiently as possible to the correct server.

The system was implemented by the author in Go language, with gRPC used for internal communication and HTTP for external communication.

The correctness of solution has been tested with multiple tests. Particular emphasis was placed on many possibilities of failure, including local failures or broader network partitions. Benchmarks show that the system achieves performance comparable to industry grade standards while maintaining its goals of data consistency.

Poniższa praca opisuje projekt i implementację systemu rozproszonego który przechowuje dane w modelu klucz-wartość. System został zaprojektowany do zarządzania danymi które zostały podzielone na wiele serwerów. Główną gwarancją i celem systemu jest to aby po każdym zapytaniu zwracał poprawne, aktualne dane kosztem krótkotrwałej niedostępności. Wiele produkcyjnych systemów, takich jak Redis, Apache Cassandra czy Amazon DynamoDB, stawia na to, aby system cały czas działał, ale pozwala na to, by w małej liczbie przypadków zwrócił niepoprawne dane. Ten projekt gwarantuje, że wszyscy użytkownicy widzą takie same dane niezależnie od serwera do którego się łączą, nawet w wypadku krótkotrwałych awarii sieci czy pojedynczych serwerów.

Architektura została zaprojektowana z podziałem na warstwy, w których każda ma określone, niezależne od siebie nawzajem cele i zadania. To pozwala na wykrywanie awarii serwerów oraz automatyczną samonaprawę systemu bez narażania integralności danych. System wspiera dedykowaną bibliotekę klienta, która optymalizuje komunikację i zapewnia efektywne wysyłanie zapytań do odpowiednich serwerów.

Rozwiązanie zostało zaimplementowane w języku Go, z wykorzystaniem protokołu gRPC do komunikacji wewnętrznej oraz HTTP do komunikacji zewnętrznej.

Poprawność została zweryfikowana testami. Nacisk został przede wszystkim położony na sprawdzanie integralności danych przy różnych scenariuszach awarii systemu. Dodatkowo zostały przeprowadzone testy wydajnościowe, które wykazują, że system osiąga wyniki porównywalne z systemami produkcyjnymi, popularnymi w branży informatycznej, a jednocześnie osiąga założone cele.

Contents

1	Introduction	9
1.1	Distributed systems and the problem with data consistency	9
1.2	Objective and scope of the thesis	10
1.3	Technology stack choice	10
1.4	Structure of this thesis	11
2	Theoretical Foundations and Overview of Solutions	13
2.1	Terminology	13
2.2	Consistency models in distributed systems	15
2.2.1	Strong Consistency (Linearizability)	15
2.2.2	Eventual Consistency	16
2.3	The CAP theorem and trade-offs	16
2.4	The Split-brain problem	17
2.5	Consensus algorithms	18
2.6	Detailed analysis of the Raft algorithm	19
2.6.1	Core concepts and terminologies	19
2.7	Overview of cache system architectures and methods	25
2.7.1	Sharding and Consistent Hashing	25
2.7.2	Role of the client: Thin Client vs. Smart Client	27
2.8	Overview of existing solutions (Redis Cluster, Etcd) - Analysis in terms of consistency	28
2.8.1	Redis Cluster	28
2.8.2	Etcd	28

2.8.3	Analysis of industry inspirations	29
3	Requirements Analysis and Software Architecture Design	31
3.1	Functional and Non-functional Requirements	31
3.2	High-Level Architecture	32
3.2.1	High-Level request flow	32
3.2.2	Separation of the Control Plane and the Data Plane	34
3.2.3	gRPC Communication Protocol and Interface Definition	35
3.3	Design of the Control Plane Module	36
3.3.1	Cluster Topology and Metadata Management	36
3.3.2	Node Failure Detection Mechanism	36
3.3.3	Sharding strategy: Fixed Shard Model	37
3.3.4	Shard Rebalancing Strategy	37
3.3.5	Data Migration and Synchronization	38
3.3.6	Split-Brain Prevention: Lease-Based Fencing	39
3.4	Smart Client Design	40
3.4.1	Epoch-Based Request Validation	40
3.4.2	Adaptive Retry Strategy	41
3.4.3	Client-Side Idempotency	41
4	Implementation of the Raft Consensus Module (pkg/raft)	43
4.1	Concurrency and Locking Model	43
4.1.1	gRPC Connection Management	44
4.2	Raft Node Roles and Lifecycle	44
4.2.1	Implementation of Roles: Follower, Candidate, and Leader	44
4.2.2	Leader Election Mechanism (election.go)	45
4.3	Log Replication Logic (replicator.go)	45
4.3.1	Isolated Replicators and the Pulse Protocol	46
4.3.2	The Log Request (AppendEntries)	46
4.3.3	Committing Entries (Commit Index) and Applying to the State Machine	46

<i>CONTENTS</i>	7
4.4 Asynchronous Persistence Management	47
4.4.1 Writing Raft State to Disk (persistence.go)	47
4.4.2 State Recovery	47
4.5 Log Compaction and Snapshotting	47
4.5.1 Snapshot Trigger and Creation	48
4.5.2 Chunked Snapshot Transfer (InstallSnapshot)	48
4.6 System Configuration and Environment Parameters	48
5 Implementation of the Control Plane and the Data Plane	51
5.1 Implementation of the Control Plane	51
5.1.1 Topology State Machine	51
5.1.2 Node Health Monitoring	52
5.1.3 Shard Rebalancing	53
5.1.4 Controller HTTP API Reference	54
5.1.5 Lease Management and Fencing	56
5.2 Data Plane	56
5.2.1 Concurrent Data Store	56
5.2.2 Synchronization and Consistency	57
5.2.3 Data Transfer Protocol	58
5.2.4 data node HTTP API Reference	58
5.3 Smart Client Mechanism	59
5.3.1 Adaptive Retry and Backoff Strategy	60
5.3.2 Client-Side Idempotency	60
5.4 Control Plane API Layer (pkg/api)	61
5.4.1 Bridging Raft and HTTP	61
5.4.2 Idempotency and Resilience	61
5.4.3 Client API Reference	62
6 Correctness verification and Fault injection testing	63
6.1 Methodology for testing distributed systems	63
6.2 Unit and integration tests for the Raft implementation	64

6.3	Simulation of failures and network partitioning	65
6.4	Continuous Integration (CI/CD)	66
7	Performance Analysis and Comparison with Existing Solutions	67
7.1	Benchmarks	67
7.2	Throughput Study (RPS) Depending on the Number of Clients and Shards	68
7.3	Analysis of Raft Protocol Overhead on Latency	69
7.4	Comparative Study	69
7.4.1	Performance Comparison with Etcd	70
7.4.2	Performance Comparison with Redis	70
7.4.3	Conclusions from the Comparative Analysis	70
8	Summary	71
8.1	Evaluation of the Achievement of Research Objectives	71
8.2	Practical use of the software	71
8.3	Directions for Further Development	72
	Bibliography	75
A	Installation	77
A.1	Project location	77
A.2	Project Structure	77
A.2.1	Prerequisites	78
A.2.2	Setup	78
A.3	Using the CLI	81
A.4	Performance Benchmarking	81

Chapter 1

Introduction

1.1 Distributed systems and the problem with data consistency

When collecting the data in the application, the volume can grow infinitely. Computers, however, have limited amount of resources due to hardware limits or law of physics. In consequence, developers can decide they need to scale their application to more than one computer. These computers are interconnected via a network, which introduces new challenges such as latency and partial failures. As the amount of data grows and the number of computers increases, so does the complexity of the system.

Some of the data might be reused more often than other. Querying the data in correct database and then fetching it can be slow, so developers often use specialized software to accelerate this operations, most often in the form of the local cache - a key-value store. However, the data can scale to such enormous size, that it can no longer be stored locally.

In that case, the distributed cache is used to balance the load between multiple servers and provide total capacity that exceeds the hardware limits of a single machine. Distributed systems solve the problem of limited local resources because they allow for the addition of more computers, whose individual resources are additive, forming a larger shared pool of memory. Furthermore, while the system is distributed across the network, each individual node is designed to handle requests concurrently to efficiently utilize its local hardware resources. To ensure data reliability and availability, these systems often store multiple copies of the same data on different nodes, known as replicas. However the data in cache is meant to be accessed and modified frequently, the conflicts between two server might arise. In most applications, this won't be a problem, because developer can simply refetch the data from database or ignore these uncommon events. However, there are systems that require the data to be correct at all times. This challenge is addressed by the problem of data consistency, which describes how the conflicts are managed and how

often they occur.

1.2 Objective and scope of the thesis

The objective of this thesis is to design and implement a distributed in-memory cache, a key-value store, that ensures strong consistency of stored data. Strong consistency implies, that at all times, system behaves as it's a single-node entity where every node agrees on the current state. The strong consistency concept will be defined in more depth later in chapter 2.2.1.

It is important to note, that due to the system being a cache rather than a database, data persistence after crash is not guaranteed. Since data is stored exclusively in-memory on the data servers to maximize performance, it remains volatile. If a server, and all replicas on which the data is stored, crashes or its host servers restarts, the data is lost and has to be refetched from persistent storage such as a database and put again into the system. The primary guarantee of this system is not data persistence after failures, but the consistency - correctness. Whenever user asks for the data and receives a response, this response will be correct and consistent across every server in the cluster.

The user interface is implemented as a command line tool that supports all key-value store operations. The system runs on Docker containers and allows user to easily start itself on a local machine. With some configuration, the user can also start it on many computers and add new servers when needed.

1.3 Technology stack choice

The Go programming language was chosen due to its advanced concurrency model. System relies on the concurrent operations both inside the node and also across the nodes in the cluster. Due to the need for a tool that supports such tasks, the Go was chosen. Furthermore, the system also uses a lot of network intensive operations, which Go excels at.

There were other considerations for languages of the project, namely C++ and Java, but both were rejected. C++ because of its high complexity and the potential impact on the development time considering author's proficiency level. Java was excluded, because it is generally slower than Go in network-intensive tasks and has a higher resource overhead, and the goal is to minimize the use of resource.

For internal communication I utilized gRPC, a high-performance framework for calling functions on other servers. The choice was made because of gRPC's high performance and efficient binary serialization. For external communication the HTTP protocol was selected due to its simplicity and ease of integration.

1.4 Structure of this thesis

Chapter 2 presents theoretical background necessary to understand the problem of state management in distributed systems. It discusses the CAP theorem, data consistency models, consensus algorithms - what are they, why we need them. It compares two of the most popular consensus algorithms and describe what the chosen one is doing. The chapter reviews existing architectural patterns across industrial solutions, techniques for splitting data between multiple servers and client types.

Chapter 3 outlines more detailed requirements of the project and the system design. It defines high-level architecture, separation of Control Plane from Data plane. It also details communication between servers and the specific design of client. This chapter is a theoretical description of the architecture of the system.

Chapter 4 presents a detailed implementation of Raft consensus algorithm. It describes implementation of basic parts of the algorithm as well as more advanced.

Chapter 5 covers the implementation details of the architecture. It describes in detail the implementation of every part of the system that was mentioned in chapter 3. Special attention is given to mechanisms of shard rebalancing and failure detection.

Chapter 6 is dedicated to testing and verification of the system. It describes the testing methodology, including unit tests, end to end tests, integration tests and failure injection tests used to validate system correctness and stability.

Chapter 7 presents the performance analysis and benchmarks in comparison to other industry-standard systems. It contains the results of benchmark tests measuring latency and throughput (requests per second) in comparison to other industry standard systems.

Chapter 8 summarizes the thesis, evaluating the degree to which the project goals were met, discussing the encountered challenges and potential directions for future development.

Appendix A provides an operational guide for the system, including environment setup and instructions for running the project locally.

Chapter 2

Theoretical Foundations and Overview of Solutions

2.1 Terminology

In this thesis, several technical terms are used to describe the components and mechanisms of the system. These terms are grouped below by their context within the distributed architecture.

System Hierarchy and Architecture

- **Node / Server / Instance** - the smallest unit of the system, referring to a single running process usually isolated in its own container or physical machine.
- **Cluster** - a collective group of nodes working together. Initially, the system operates as a single cluster containing all nodes. However, due to network failures or partitions, the cluster can be split into multiple independent **sub-clusters** that are unable to communicate with each other.
- **Cache / Key-Value Store / System** - the entire distributed entity designed and implemented in this project, encompassing all nodes and their configurations.
- **Control Plane / Data Plane** - the architectural separation of nodes, between the management layer (**Control Plane**) and the actual storage layer (**Data Plane**). This will be in depth more described in chapter 3 and 5.

Roles and Replication

- **Primary / Replica** - roles within servers that store the same data; the **Primary** node handles writes and ensures consistency by replicating data to one or more **Replica** (Backup) nodes. Replica is just a copy of the data in case the primary node crashes.

- **Synchronous Replication** - a method where the system waits for all replicas to save the data before telling the user it succeeded. This is slow but safe.
- **Asynchronous Replication** - a method where the system tells the user "OK" immediately after the primary saves data, without waiting for replicas. This is very fast but can lose data if someone crashes.
- **Quorum / Majority** - refers to more than half of the total nodes in a cluster (e.g., 3 out of 5), required to reach a consensus, as explained later in section 2.5.

Internal Mechanisms and Concurrency

- **RPC (Remote Procedure Call)** - a way for one server to call a function or send a message to another server over the network.
- **Epoch** - a version number for the cluster topology (what is a topology will be defined later in chapter 3); it increases whenever the system configuration changes.
- **Reaper** - a background process that monitors nodes and "reaps" (removes or handles) those that have failed. Described in more detail in chapter 3.
- **Goroutine** - a lightweight, independent task managed by the Go programming language that allows the system to do many things at the same time.
- **Channel** - a "pipe" used by goroutines to communicate and send messages to each other safely.
- **Mutex (Lock)** - a mechanism that ensures only one part of the program can access or change a piece of data at a time to prevent errors. There are different types: a regular mutex allows only one accessor (reader or writer), while an RWMutex (Read-Write Mutex) allows multiple readers but only one writer at a time, improving performance when reads are more common than writes.
- **Thread Safety** - ensuring that multiple goroutines can access shared data at the same time without causing corruption, conflicts, or incorrect results. This is achieved using locks, channels, or other synchronization mechanisms.
- **Deadlock** - a bug where two or more parts of a program are waiting for each other to finish, causing the entire system to freeze.
- **Ticker** - a repeating timer that triggers an action at regular intervals. For example, a 50ms ticker will trigger something every 50 milliseconds.
- **Serialization** - the process of converting data from memory into a format that can be saved to disk or sent over a network. Deserialization is the reverse converting saved or received data back into memory. Common formats include

Protocol Buffers, JSON, and binary encoding - for example with the help of Go's gob encoding library.

- **Durability** - feature of the system that data survives crashes and restarts, because it has been saved to disk (or other stable storage), not just kept in memory which is lost on failures
- **Modular Design** - breaking a complex system into smaller, independent pieces (modules) where each piece does one specific job. This makes the system easier to understand, test, and modify.

Performance Metrics

- **Throughput** - the total number of operations a system can perform in a given time (e.g., 1000 requests per second).
- **Latency** - the time it takes for a single request to travel from the client to the server and back.

2.2 Consistency models in distributed systems

Distributed systems operate on data. Consistency is a metric that describes how many conflicts between nodes are resolved and how long it takes to resolve them. In this thesis, I will focus on two main categories of consistency models: strong and weak.

2.2.1 Strong Consistency (Linearizability)

This is a strongly consistent model that ensures that all nodes agree on the order in which operations were done. Reads will always return the most recent version of data. When an update occurs, this model makes sure that every other server reflects that change. Due to these guarantees, this model limits how fast the system can work, but allows the system to be seen as a single-node entity.

Linearizability is the strongest guarantee for consistency for single-object requests (operations like getting or setting a single key). It gives an illusion that the system is in fact a single copy of data and the data is not distributed. It guarantees that once a write operation is completed successfully, all subsequent read operations will return that value, or a newer value if any update happened.

Linearizability (the formal definition of Strong Consistency) says:

- If operation A completes (client gets OK), then every operation B that starts after A finishes must see the effect of A.

- If an operation A is in-flight or timed out, it is allowed to either "have happened" or "not have happened," as long as the system is consistent about that choice from then on.

Mathematically, when operation $Write(x)$ completes at time t_1 , then for any given operation $Read(x)$ starting at time $t_2 > t_1$ must see the effects of $Write(x)$.

2.2.2 Eventual Consistency

This is one of weak consistency models that prioritizes availability and performance. It allows data between servers to conflict, but if no updates occur, eventually these conflicts will resolve. In eventually consistent systems, if no new updates are made to a given data item, eventually all accesses to that item will return the last updated value. This model allows for temporary inconsistencies where different nodes serve different versions of data. This approach enables high availability and low latency because writes can be accepted by a single node without waiting for other nodes to accept and agree of the state of the system. This approach, however, requires adding logic for conflict resolution strategies (rules to decide which data is correct, for example, "the newest write wins") when merging data from different nodes. One example can be a like system on a social media platform. Imagine we have two friends, one who lives in Australia and the second one who lives in Poland. Before the Australian friend likes a post, the number of likes is x , but after the Australian friend likes the post, the number of likes is $x + 1$. The Polish friend however, still sees the amount of likes equal to x . Only after some small amount of time, the number of likes synchronizes to $x + 1$. This is a temporary inconsistency, that the eventual consistency allows. The number of likes is not a crucial data that can cause any issues if it is not consistent, hence system can allow for temporal inconsistencies.

Having explored both strong consistency (where all nodes always agree) and eventual consistency (where nodes may temporarily disagree), a natural question arises: why do these different models exist? Why can't a distributed system simply provide both perfect consistency and perfect availability at all times? The answer lies in a fundamental theoretical limitation of distributed computing, known as the CAP theorem.

2.3 The CAP theorem and trade-offs

CAP theorem, formulated by Eric Brewer in [1], says that a networked system that shares data, can strictly provide only two of three properties:

- **(Strong) Consistency (C):** Reads will receive the most recent write or an error. If a system is consistent, then all nodes will see the same data at the same time, just like a single-node database would.

- **Availability (A):** Request will always receive a non-error response, without the guarantee that it can contain conflicting data. Availability says that the system remains operational for reads and updates, even if some nodes fail.
- **Partition Tolerance (P):** The system will work even if a network failure occurs or some nodes crash or there are very big delays. The system continues to operate despite an arbitrary number of messages being dropped or delayed by the network between nodes.

In real-world distributed systems, we cannot have Consistent and Available system, because the distributed system itself is a partitioned system, so for it to work it has to be partition tolerant. For example if a hardware failure or network failure occurs, then there is inevitable Network Partition. Hence in real-world distributed systems, the choice is between Consistency and Availability. Most cache systems focus on Availability, because authors want to maximize performance. Most cache operate on argument, that if a user wants consistency, they should use database. However, there are cases, where the need of Consistency is more important than Availability, for example in banking applications or in any system where data correctness is critical.

The distributed systems are either:

1. **CP ((Strong) Consistency + Partition Tolerance):** When partition occurs, system will shut down all non-consistent nodes or refuse writes if the system cannot support them any longer. This approach prioritizes safety over throughput or uptime.
2. **AP (Availability + Partition Tolerance):** When a partition occurs, system will continue to accept write and read requests, with probability that it will return conflicting data. This approach prioritizes throughput, uptime and user-experience over correctness of the data.

In this thesis, I have chosen to implement a CP system, prioritizing data integrity as the primary requirement for the distributed cache.

2.4 The Split-brain problem

There are many problems that can occur due to splitting application from one servers into multiple. One of this problem is a Split-brain problem, that this system too has to solve.

Split-brain is a problem in distributed system where a network divides the cluster into two or more disconnected groups (sub-clusters). Due to disconnection, the nodes cannot communicate with each other, so each node may incorrectly deduct

that other nodes in the other group have died. This creates a situation where separate parts of the system operate independently and accept conflicting writes, believing they are the only active group. There are many strategies to manage this problem. Algorithms like Raft only allow a node to become a leader (definition of leader will be explained in next section) if more than half of total nodes agrees. Another solution to this problem is called fencing - a technique that makes a node physically or logically disable its peer to ensure it cannot write to shared storage.

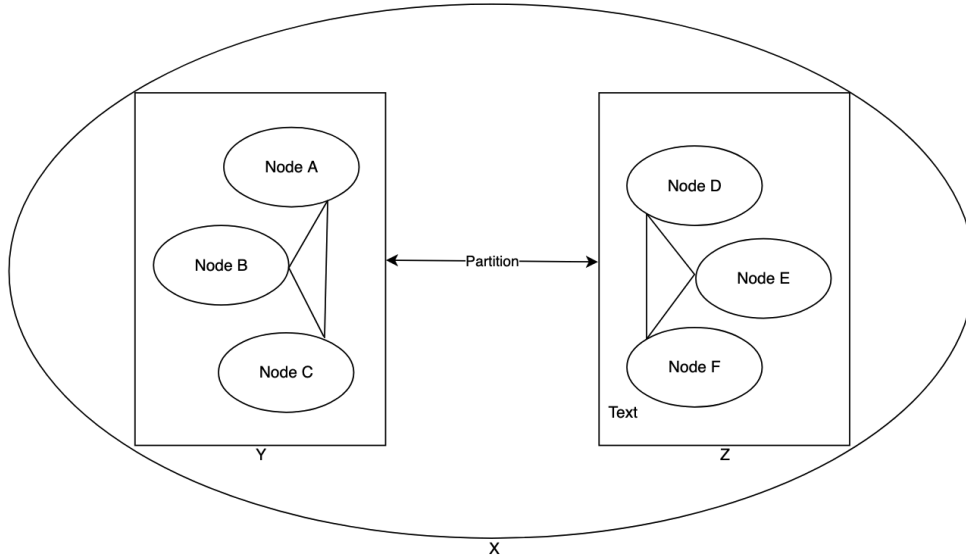


Figure 2.1: Visualization of split-brain problem. The network X has been split into subnetworks Y and Z

The system has to decide what to do when split-brain problem occurs, but that will be described more in chapter 5. However, there already comes a conclusion, that we need some mechanism that disallows any damage this problem can do. Consensus algorithms are one of the possibility to handle split-brain problem.

2.5 Consensus algorithms

A consensus algorithm is a set of rules that allows all servers in a system to reach a single agreement on the same data or sequence of actions. This ensures that every server has exactly the same information at any given time, even if some of them crash or have network problems. It serves as the backbone of the system I developed for this thesis - the Control Plane is entirely based on it. There are two very popular algorithms: Paxos and Raft.

Paxos is a family of protocols created by Leslie Lamport [14] and historically the most used algorithm for distributed consensus. While it is mathematically

proven to be safe and efficient, it is also very complex to understand and difficult to implement in real-world systems.

Raft is a modern algorithm, introduced in 2014 by Diego Ongaro and John Ousterhout [2], that was designed to be easier to implement than Paxos and more understandable than it. It offers performance comparable to Multi-Paxos [15] (a more efficient version of the original Paxos). It decomposes the problem of consensus into smaller, independent sub-problems of choosing a coordinator, replicating the sequence of updates and ensuring safety. Raft is widely used in many modern distributed systems such as CockroachDB [3], MongoDB [4], Neo4J [5], RabbitMQ [6].

Due to its focus on implementability and the availability of robust documentation, I have selected the Raft algorithm as the foundation for this project. Next section will describe Raft in more details.

2.6 Detailed analysis of the Raft algorithm

2.6.1 Core concepts and terminologies

Before describing how the algorithm works, it is important to note, that it uses several key concepts. In consequence, understanding these terms is necessary to follow the detailed analysis. For more details, both about definitions and overall Raft working idea, reader should refer to the original Raft paper [2].

- **State Machine** - A state machine is the application logic that processes commands and maintains state (data). In a distributed system using Raft, each server has its own local state machine. The fundamental goal of the Raft algorithm is to ensure that all servers' state machines execute the same sequence of commands in the same order, reaching identical states. For example, in a key-value store, the state machine would handle PUT and DELETE operations, and Raft ensures that all replicas agree on which operations occurred and in what order.
- **Leader** - Only one server can be a coordinator of the data at any time. This server is called a leader, and all other servers must treat it as a source of truth. It manages all client requests and coordinates rest of cluster.
- **Elections** - Since system needs a leader to function, there must be a way to pick one. If current leader stops working, nodes start an election to choose a new one.
- **Logs** - A log is a sequence of entries that store operations sent by users. We need them to keep track of what happened in the system so it can be recovered later. Logs represent exact order of the operations history.

- **Term** - This is a logical clock. It acts as a version number for the Raft algorithm, helping nodes to identify which information is current and which is old.
- **Safety** - Safety is a set of rules that guarantee that nodes never agree on different things for the same operation. It also guarantees that node who crashed and rebooted will not become a leader until the data it has is up to date.
- **Commit Index** - The index of the highest log entry known to be committed (i.e., stored on a majority of nodes). Once an entry is committed, it is considered permanent and will eventually be applied to all state machines in the cluster.

The Raft algorithm is designed to manage a replicated log on behalf of a state machine, ensuring that all nodes in a cluster eventually agree on the same sequence of operations. One of its fundamental properties is its fault tolerance: Raft ensures that a cluster of N servers can tolerate F failures, where $N = 2F + 1$. This means a majority of nodes must be functional for the system to remain available and consistent.

It is standard practice to use an odd number of nodes in a cluster because even-sized clusters do not provide additional fault tolerance. For any $N = 2n + 1$ (odd) and $N = 2n + 2$ (even), both clusters tolerate exactly n failures. A quorum Q is defined as $\lfloor N/2 \rfloor + 1$. For $N = 3$, $Q = 2$ and the system tolerates $3 - 2 = 1$ failure. For $N = 4$, $Q = 3$ and the system also tolerates only $4 - 3 = 1$ failure. Consequently, a 4-node cluster is less efficient than a 3-node cluster; it requires more resources and a larger number of servers to reach an agreement, while providing the same level of reliability.

Having understood that Raft requires Quorum to work correctly, the practical question that comes to mind is how is the leader selected. Next section will broadly describe this, but for more details one should read original paper.

Leader Election

Raft defines three states a node can be in - Follower, Candidate and Leader. The Leader is responsible, as in name, for leading the cluster. It accepts the requests, it sends the data to different servers. The Leader is the coordinating server of all of the other servers in cluster. Followers always listen to the Leader. The Candidate is a role that Follower transitions to, when he wants to become a Leader when election starts.

Raft uses a heartbeat mechanism (an "I am alive" signal) to maintain leadership. If no heartbeat is received in some configurable time window, then server can assume the leader is dead and start a new election.

The election starts by the node incrementing its current **Term** (a logical clock used to detect stale information), changing its current state to the Candidate state, voting for itself, and broadcasting **RequestVote** RPCs to all other nodes. A candidate wins if it receives votes from a majority of the cluster.

To prevent conflicts during elections, Raft uses randomized election timeouts. Each node waits for a random amount of time before starting an election. This randomization ensures that in most cases, only one node will timeout first and become a candidate, avoiding split votes where multiple nodes compete simultaneously and no one achieves a majority. Without this randomization, multiple nodes could start elections at the same time, leading to repeated failed elections and system stalls.

For a complete specification of the leader election algorithm, including detailed edge cases and safety proofs, readers should refer to the original Raft paper [2].

The cluster is now ready to accept the requests, but system is using multiple servers, so the servers need to exchange information. Log replication is used for that.

Log Replication

Once elected, the leader accepts client commands. A command flows through the following lifecycle:

1. The leader appends the command to its local logs.
2. It sends **AppendEntries** RPCs to all followers.
3. Once a majority of followers acknowledge the entry, the leader marks the entry as committed.
4. The leader applies the entry to its state machine and returns the result to the client.
5. In subsequent heartbeats, the leader notifies followers of the new **commitIndex**. This is the highest index known to be safely replicated on a majority of nodes; once followers receive this value, they apply all entries up to that index to their own state machines.

Raft uses a simple but powerful principle: if the logs between two servers are identical at some index (both the index positions and the values match), then their entire prefix is also identical. This property enables an efficient optimization where the leader only needs to find the last point where its log matches the follower's log, and then send the suffix (the remaining entries) from that point forward.

Example of diverging logs after a network partition:

Consider a cluster with 5 nodes (A, B, C, D, E) where Server A is the leader in Term 1:

1. Initial state: All servers have log entries [1, 2, 3] from Term 1
2. A network partition splits the cluster: (A, B) on one side and (C, D, E) on the other
3. Before the partition is detected, the old leader A receives two client requests and appends them locally: A's log becomes [1, 2, 3, 4, 5]
4. Since A cannot reach a majority (only has B), these entries are never committed while the clients are waiting for acknowledge
5. The other partition (C, D, E) elects a new leader C in Term 2
6. Leader C receives different client requests and successfully commits them with majority: C's log becomes [1, 2, 3, 6, 7] where 6 and 7 are from Term 2
7. The network partition heals
8. Leader C (with higher term) sends **AppendEntries** to A
9. A detects that C has a higher Term, steps down to Follower state
10. C finds that logs match up to index 3 (the prefix [1, 2, 3])
11. C instructs A to delete uncommitted entries [4, 5] and replace them with the correct suffix [6, 7] from Term 2
12. A's log is now [1, 2, 3, 6, 7], matching the cluster consensus

This example demonstrates why the term number is crucial: entries from an old term (Term 1) that never reached a majority are overwritten by committed entries from a newer term (Term 2). This ensures safety.

Once Leader replicates the log to the majority of nodes (at least a quorum) it will apply the operation to the local state machine.

However, the system is working in distributed network, so many problems can occur. One of them is where leader loses connection for some time and cluster operates in the meantime. Raft introduces a Safety mechanism for that case and some other problems.

Safety

Raft ensures safety via the Election Restriction. Candidate cannot win election if its log is not in the most up to date state. During the voting process, a voter denies its vote if the candidate's log is less up to date than its own by comparing the index

and term of the last log entry. Additionally nodes utilize a Term number to detect and reject stale leaders. If a node receives a request from a leader with a lower term than its own, then it will reject that request and force a stale leader to step down. This mechanism guarantees the State Machine Safety property, that is, if a server has applied a log entry at given index to its state machine, then no other server will ever apply a different entry at the same index. This guarantees the prefix is also the same.

Persistence

The safety guarantees described above only work if nodes can remember their state across crashes and restarts. To achieve this, Raft requires each server to persist four critical pieces of information to stable storage (typically hard drive) before responding to any RPC:

- **currentTerm** - The latest term the server has seen. This prevents a restarted node from participating in an old election with a stale term.
- **votedFor** - Which candidate received this server's vote in the current term. This ensures a server doesn't vote twice in the same election after a crash.
- **log[]** - The entire sequence of log entries. This allows a restarted node to know which operations have been replicated and ensures the log history is never lost.
- **committedLength** - The index of the highest log entry known to be committed. Persisting this allows the node to know which entries can safely be applied to the state machine after a restart.

Due to log growing infinitely, the Snapshot mechanism is introduced, that is also saved to persistent storage. Snapshot mechanism will be described in next section.

Without persistence, a node crash or restart might damage the safety of the entire system. Consider the following dangerous scenario:

Example of failure without persistence:

1. Server A votes for Candidate B in Term 5 and updates its **votedFor** field to B
2. Before saving this information to disk, Server A crashes
3. Server A restarts with no memory of having voted
4. Another Candidate C requests votes for the same Term 5
5. Server A, having lost its vote record, votes for C as well

6. Now both B and C might receive a majority, potentially creating two leaders in the same term, violating Raft's safety guarantees

By saving the data to disk, Raft ensures that the most crucial data is not lost after any failure. When Server A restarts in the example above, it reads from stable storage that it already voted for B in Term 5, and correctly rejects C's vote request. Server always recovers its state from the stable storage and by design does it quickly. The detailed implementation of this will be described in Chapter 4.

Log compaction and Snapshotting

Infinite log growth causes many problems. First, the storage requirements for maintaining complete logs can become very large. Second, transferring large logs across the network (potentially millions of entries) may cost too much. Third, recovery time after a crash becomes increasingly problematic as the server must read entire log into memory one by one (because the order of the log is very important) to maintain correctness, resulting in extended downtime.

An important observation reveals that not all log entries are necessary to maintain system state. Consider an example where operation *X* performs `PUT 1 3` (setting key 1 to value 3), and a subsequent operation *Y* (where *Y* has been executed after *X*) performs `DELETE 1` (deleting key 1). Once all servers reach consensus on this state, the value of key 1 no longer exists in the key-value store, yet the log keeps this obsolete information. Similarly, when a key is changed multiple times, only the final value is useful to the system.

To address these problems, the snapshot mechanism is introduced. When the log size exceeds a configurable threshold, the current state of the cache (all key-value pairs) is serialized and saved to a snapshot file. In the worst case, where each operation modifies a distinct key, the snapshot size equals the number of operations. However, this scenario is highly unlikely in practice, as operations typically modify a limited set of keys repeatedly. Consequently, the snapshot size is generally significantly smaller than the complete log.

Snapshot recovery is straightforward: the system restores its state by loading the key-value pairs from the snapshot file. This recovery process can be performed in parallel, as each key has a single associated value and there are no dependencies or conflicts between different key-value pairs.

This approach introduces a new problem - alongside of logs replication, the leader has to replicate snapshots too. However, as noted previously, snapshots guarantee to be smaller than the real logs would be, so this is more efficient than log replication. The Leader has to replicate snapshot if a follower is so far behind that the Leader has already discarded the needed log entries to find its latest index where these two servers agree to the prefix of the logs.

After establishing the consensus foundation via Raft, the next consideration is how to organize data across multiple servers and how clients interact with the distributed cache.

2.7 Overview of cache system architectures and methods

This section explores common architectural patterns for sharding strategies and client designs, which form the basis for the implementation decisions discussed in later chapters.

Consensus algorithm ensure the coordination of servers is correct, but architectural design is what allows the system to scale to handle gigabytes of data and thousands of requests per second. One of the first problems to consider is how to distribute data across multiple servers.

2.7.1 Sharding and Consistent Hashing

To distribute the data between multiple servers, the process called sharding is introduced. Sharding is a process of partitioning data into multiple segments, each stored on a different server.

Due to the need of deciding which server should store which piece of data, we use hashing. A hash is a function that takes a key and turns it into a specific number. If we calculate a hash for a key and the result points to server X, then the data for this key is placed there. In consequence, the system can quickly know where the data is, without asking every server in the cluster.

Below are two examples of commonly using hashing methods.

Traditional Modulo Hashing

The most basic approach would use $server = \text{hash}(key) \pmod{N}$, where N is the number of servers. This method is fine in static environments where the number of servers remains constant. However if the system needs to scale dynamically, changing N causes the result of modulo operation to change nearly all keys in the system. This is highly ineffective.

Consistent Hashing

Consistent hashing maps both data and servers onto a logical ring topology (0 to $2^{32} - 1$). A key is assigned to the first server encountered moving clockwise on the

ring. [9]

When a server is added or removed, only K/N keys need to be remapped (where K is the total keys), which significantly minimizes data movement.

When some part of the ring is used way more often than other we use Virtual Nodes: each physical server is placed onto the ring multiple times (e.g., at 100 different positions). This statistical distribution ensures a more uniform load balancing across the cluster.

These are basic ideas of hashing. Below there are some industrial used hashing and sharding techniques.

Karger's Consistent Hashing

Proposed by Karger et al. at MIT in 1997 [9], consistent hashing solves the reshuffling problem inherent in modular hashing. In a standard modulo setup ($\text{hash}(k) \pmod n$), changing n (adding/removing a node) results in nearly all keys being remapped. Karger's approach maps both the Nodes and the Keys onto the same identifier space (a ring).

Why it was not used: While industrially popular (consistent hashing is used in Cassandra [13] and Dynamo [10]), it introduces complexity in the Control Plane metadata management. Maintaining a sorted ring and handling virtual nodes requires more complex logic than a fixed partition map. For this thesis, I opted for a Fixed Slot approach (similar to Redis) because it allows for deterministic shard-to-node mappings that are easier to synchronize via Raft.

Range Partitioning (Sorted Monolithic Map)

Range partitioning is a data distribution strategy where they treat the entire key space as one sorted collection. For two servers, the idea would look like - server one stores all keys from A to K, where server two stores the keys from L to Z. In this approach, keys are stored in alphabetical or numerical order, and the data is into ranges. Each server is responsible for its segment of the sorted key space.

- **Pros:** It allows for efficient range queries - the system can easily query which servers will have which keys and efficiently go to these servers. It allows to query only one server that is responsible for key that the user is interested in.
- **Cons:** The major disadvantage is the Sequential Write Problem. Think of storing logs with timestamps. Because the keys are split into ranges, then there might be a situation where server one will accept all keys upto July and the second server will accept all keys starting in July. This creates the problem where some servers might be underutilized where others might be overwhelmed.

Why it was not used: Cache primary purpose is to handle specific key lookups efficiently and not ranges. If the system wants only a specific key, having the sorted order of keys doesn't provide any advantage. Range partitioning is better suited for analytical systems, where users frequently run queries that fetch data from some time range or with some very specific parameter.

FNV-1a Hashing Algorithm

The Fowler-Noll-Vo (FNV) hash 1a [12] is a non-cryptographic hash function known for its extreme speed and ease of implementation. It works by multiplying the hash by a large prime and XORing it with each byte of the input data.

- **Pros:** High throughput, low collision rates for short strings (ideal for cache keys), and minimal memory overhead.
- **Why it was used, why it suits the project:** Since this project requires mapping keys to a fixed set of shards during every client request, a low-latency hash function like FNV-1a is preferable over more complex cryptographic hashes like SHA-256 or MD5, which would introduce unnecessary CPU overhead.

The system knows how to store the data and how to access it, but the client does not. There are two main options of clients, thin and smart.

2.7.2 Role of the client: Thin Client vs. Smart Client

A crucial decision is on how the client interacts with the system:

- **Thin Client (Proxy-based):** The client is unaware of the cluster topology. It needs help from the system, so the system will only share some point to where client sends the request and the system will handle the rest.
 - *Pros:* Simple client implementation; topology changes are transparent to the application.
 - *Cons:* Additional network hop (Client → Proxy → Shard), increasing latency.
- **Smart Client:** The client maintains a local map of the cluster topology (slots mapping to node IP addresses). It computes the hash and connects directly to the target node.
 - *Pros:* Lowest possible latency (single hop); no bottleneck at a central proxy.

- *Cons:* Complex client logic; the client must listen for cluster configuration changes (e.g., failovers or resharding) to update its internal routing table.

In this thesis, I have implemented a Smart Client to improve the latency time and for scalability.

2.8 Overview of existing solutions (Redis Cluster, Etcd) - Analysis in terms of consistency

This section compares two distributed key-value stores - Redis Cluster and Etcd, analyzing their architectural choices and consistency guarantees to provide context for the design decisions made in this thesis.

2.8.1 Redis Cluster

Redis Cluster is the industry standard for distributed caching. It employs a multi-master architecture (where multiple nodes act as "masters" — primary nodes that have the authority to accept and process write operations for their specific subsets of data) with asynchronous replication.

- **Architecture:** It uses sharding with 16,384 hash slots. Each master node owns a subset of slots and has one or more replicas.
- **Consistency:** Redis prioritizes Availability and Performance (AP-leaning). By default, replication to slaves is asynchronous. If a master crashes after accepting a write but before replicating it to a slave, and that slave is promoted to master, the write is lost. Redis does not guarantee strong consistency; it provides eventual consistency and high throughput.

2.8.2 Etcd

Etcd is a distributed key-value store used primarily for shared configuration and service discovery (e.g., in Kubernetes).

- **Architecture:** It is built strictly on the Raft consensus algorithm. Unlike Redis, it does not shard data by default; every node replicates the full state machine.
- **Consistency:** Etcd is a CP system guaranteeing strict serializability. Every write goes through the Raft leader and must be committed to a quorum before returning success. This ensures data safety at the cost of lower write throughput compared to Redis.

2.8. OVERVIEW OF EXISTING SOLUTIONS (REDIS CLUSTER, ETCD) - ANALYSIS IN TERMS

- **Durability:** Etcd is a system that guarantees durability: Even if a node crash, the data is not lost.

The solution implemented in this work aligns more closely with the Etcd model regarding consistency (using Raft for strong guarantees) but incorporates architectural elements of Redis (key-value storage focus) to explore the performance implications of strong consistency in a caching scenario. My solution **does not** guarantee durability, focusing instead on in-memory performance while maintaining strong consistency.

There are more tricks that industrial applications use and I have been inspired by. Below section describes them.

2.8.3 Analysis of industry inspirations

This subsection describes some architectural patterns from production systems that influenced the design choices in this work.

Redis Cluster: The "Fixed Shards" Inspiration

Redis Cluster pre-shards the key-space into 16,384 "hash slots". A key is mapped to a slot using a mathematical formula called **CRC16**: $\text{slot} = \text{CRC16}(\text{key}) \pmod{16384}$ (where modulo gives the remainder of the division).

- **Inspiration:** My implementation adopts this "Fixed Shard" model. By mapping keys to a constant number of shards (e.g., 10 or 1024) the system decouples the data distribution from the number of physical nodes.
- **The Difference:** Redis uses asynchronous replication by default to maximize performance, which can lead to data loss. This system uses Synchronous Replication to ensure Strong Consistency, sacrificing a degree of write latency for the safety guarantees required by the thesis.

Amazon Dynamo: Availability vs. Consistency

Dynamo is the archetypal "AP" (Available, Partition-Tolerant) system. It uses consistent hashing and eventual consistency mechanisms like Vector Clocks (to track versions of data) and a Gossip protocol (where servers share updates with each other like neighbors sharing rumors) to ensure it is "always on" [10].

- **Inspiration:** Dynamo's concept of a "Preference List" for replication was an inspiration for how the Controller manages shard assignments.

- **The Difference:** Dynamo’s philosophy is ”eventual consistency.” It allows conflicting writes and resolves them later. This system purposefully takes the opposite path: identifying as a ”CP” system that uses Raft and fencing to prevent conflicts from ever occurring, matching the strict requirements of linearizability.

TiKV: Raft-per-Shard vs. Centralized Control

TiKV is a modern distributed key-value store where every ”Region” (shard) is its own independent Raft group [11].

- **Comparison:** TiKV’s approach is the gold standard for massive scalability. However, running 1000 shards would mean running 1000 independent Raft state machines per node, which is computationally expensive.
- **My Approach:** To balance performance with safety, I separated the system into a Control Plane (Raft) and a Synchronous Data Plane. This significantly reduces the overhead; only the cluster metadata (topology) requires the full weight of the Raft algorithm, while data replication is handled by a simpler, faster synchronous Primary-Backup protocol (where the primary just forwards the data directly to its replicas).

Chapter 3

Requirements Analysis and Software Architecture Design

This chapter presents the requirements of the entire system. It describes what user expects from the system and what system guarantees.

3.1 Functional and Non-functional Requirements

Functional requirements define specific behaviors and functions the system must provide. In the context of this thesis, I have established the following functional requirements:

- Support GET (reading), PUT (saving), and DELETE (removing) operations via an HTTP API (a standard way for different software to talk to each other over a network).
- Automatic sharding of the keyspace across a configurable number of shards.
- Automatic replication of each shard (primary + replicas).
- Dynamic cluster membership - data nodes can be added and removed at runtime.
- Automatic node management when nodes join, leave, or fail.
- A Smart Client that routes requests to the correct node and caches the topology.

Non-functional requirements define how the system operates and what it guarantees in terms of performance, security, usability, and reliability:

- Strong Consistency - the system must reject requests when a quorum cannot be reached; the system guarantees that all successful responses are correct.

- Partition Tolerance - the system continues to operate even if some of the nodes fail or are disconnected from others.
- Fault Tolerance - the system detects node failures and triggers automatic failover - the process where a healthy server takes over the work of a failed one.
- High Availability during maintenance - no downtime when adding or removing nodes and redistributing shards between data nodes.

Now that the system requirements are known, the next section will describe high level architecture of a system.

3.2 High-Level Architecture

The system, that is cluster of servers, is split into two logical parts: the Control Plane and the Data Plane. Some of the servers are assigned to be part of the Control Plane, while others are assigned to the Data Plane. The part of architecture into which the server is assigned will dictate how it will behave and what it will do.

Topology is defined as current configuration and state of the nodes of the entire system - which nodes exist, what's their status - failed or active, which shards they are responsible for and which role they play (primary or replica) for each shard. The topology includes an Epoch counter that says how many times the topology has changed. Epoch counter allows to logic about many things about the system, in particular it may detect incorrect requests if the epoch number is too old and make the client refetch the topology.

3.2.1 High-Level request flow

Before diving into the detailed architecture, it is helpful to understand the complete flow of a typical client request through the system. This high-level overview shows how the three main components: Client, Control Plane, and Data Plane, work together.

Step 1: Client Determines Target Shard

When a client wants to perform an operation (GET, PUT, or DELETE), it first calculates which shard is responsible for the requested key:

1. The client hashes the key using hashing
2. It computes the shard ID by taking the hash modulo the total number of shards

3. It looks up the primary node for that shard in its locally cached topology

The client maintains a local copy of the cluster topology (which nodes exist, which shards they own), so it can make this determination independently without contacting the Control Plane for every request.

Step 2: Control Plane Provides Topology (Initial Setup)

Before the client can route requests, it must first obtain the cluster topology from the Control Plane:

1. The client contacts any Controller node in the Control Plane (there are multiple Controllers running Raft consensus and the client can access all of them)
2. If the contacted Controller is not the leader, it forwards the topology request to the current leader
3. The leader responds with the complete topology: which Data Plane nodes are alive, which shards are assigned to which nodes, and the current Epoch (version number)
4. The client caches this topology locally to avoid repeated lookups

The Control Plane continuously monitors the Data Plane nodes via heartbeats. If a node fails, the Control Plane automatically promotes a replica to become the new primary and increments the Epoch. Clients detect stale topology when their requests are rejected due to an outdated Epoch and the client is then made to refetch the topology from Control Plane.

Step 3: Data Plane Processes and Replicates the Request

Once the client knows the target primary node, the request flows through the Data Plane:

1. The client sends the request to the primary node for the shard
2. The primary node validates the request (checks that it is indeed the current primary, verifies the client's Epoch is current)
3. For write operations, the primary **synchronously** replicates the data to all replica nodes:
 - The primary sends the same operation to each replica
 - Each replica saves the data and acknowledges back to the primary
 - The primary waits for **all** replicas to confirm before proceeding
4. Only after all replicas have confirmed, the primary acknowledges success to the client

5. For read operations (GET), the primary can respond immediately from its local state without any replicas interaction

This synchronous replication ensures strong consistency: if a write succeeds, the data is guaranteed to be on all nodes, so any subsequent read (even if it hits a newly promoted replica) will see the latest value.

Summary of the Flow:

- **Client** → calculates shard, looks up primary in cached topology
- **Control Plane** → provides topology on demand, monitors health, triggers rebalancing
- **Primary (Data Plane)** → validates request, replicates to all replicas, then acknowledges client
- **Replicas (Data Plane)** → receive and save the data, confirm to primary

The following sections will go into more details of each component design.

3.2.2 Separation of the Control Plane and the Data Plane

The Control Plane is responsible for:

- Running a Raft consensus algorithm that stores cluster metadata: topology, shard mapping, node status, and an epoch counter. Epoch is used as a counter for which "version" of application is currently running. The version changes when rebalancing occurs, for example when some nodes fail and replica becomes a primary node.
- Exposing internal HTTP endpoints used by data nodes for pull based shard migration.
- Performing node failure detection (via the 'Reaper' - a background process that checks if nodes are still alive) and initiating rebalancing.

The Data Plane is responsible for:

- Storing the actual key-value data.
- Handling client RPCs (GET, PUT, DELETE).
- Reporting heartbeats (regular "I am alive" signals) to the Control Plane.
- Replicating data across replicas to ensure strong consistency and fault tolerance.

Synchronous Primary-Backup Replication: Unlike the Control Plane which uses Raft, the Data Plane employs a simpler but effective replication strategy. Each shard has a designated primary node and one or more replica nodes. When a client writes data, the primary node synchronously replicates the write to all replicas before acknowledging the client. This means the primary waits for all replicas to confirm they have saved the data before returning success to the client. This approach ensures strong consistency. If a write succeeds, all nodes have the data, guaranteeing that subsequent reads will always see that write. The tradeoff is that writes are slower than asynchronous replication (where the primary acknowledges immediately without waiting), but this is necessary to meet the system’s strong consistency requirement.

To minimize the time for RPC calls in internode communication, gRPC is used.

3.2.3 gRPC Communication Protocol and Interface Definition

gRPC is a high-performance Remote Procedure Call (RPC) framework developed by Google. It uses Protocol Buffers as its interface definition language (a set of rules for how messages should look) and message serialization format (the process of turning data into a compact binary format that can be sent over the network). This provides an alternative to traditional JSON over HTTP, which results in smaller payloads (the actual data being sent) and faster processing.

The system utilizes a dedicated gRPC service for the Control Plane’s Raft operations. We define six core methods and ten data types in the `raft.proto` file - a configuration file for Protocol Buffer to automatically generate Go code that defines all structs and functions:

Table 3.1: gRPC Service Methods for Raft Consensus

Method	Responder	Purpose
Forward	Leader	Forwards a client write request to the Raft leader.
ForwardGet	Leader	Forwards a client read request to the Raft leader.
LogRequest	Follower (Non-Leader)	Standard Raft <i>AppendEntries</i> for log replication.
VoteRequest	All	Standard Raft <i>RequestVote</i> for leader election.
Heartbeat	All	Basic reachability check used in health monitoring.
InstallSnapshot	Follower (Non-Leader)	Transfers chunked state machine data to lagging nodes.

These functions form the backbone of the Control Plane’s consistency. Detailed

implementation of how these RPCs interact with the Raft state machine is discussed in Chapter 4.

As the Control Plane is the main part of the entire system, below section will broadly describe how it works.

3.3 Design of the Control Plane Module

As previously mentioned, the Control Plane is mainly responsible for storing the cluster topology.

3.3.1 Cluster Topology and Metadata Management

Topology data includes:

- Nodes map: node ID mapped to node metadata (address, status, last heartbeat timestamp).
- Shards map: shard ID mapped to shard metadata (primary ID, replica ID, status).
- Epoch: a monotonically increasing identifier used for conflict free updates.

The Controller updates the cluster topology using the Raft algorithm. I want to ensure that the Control Plane provides strong consistency, because if it did not, then the system as a whole would not be strongly consistent. This is because, if there were conflicts in the Control Plane, a client might send a request to the wrong data node, hence creating an inconsistent state which would fail to guarantee a "correct response on every request".

Rebalancing uses the Control Plane's active nodes to determine which node should be primary and which should be a replica. It then writes the new configuration to the Raft Log, which is distributed among every other Controller in the Control Plane.

However, due to working in a distributed environment and Control Plane managing topology, there must be some mechanism to detect failures.

3.3.2 Node Failure Detection Mechanism

I implemented a Reaper mechanism which tracks the last heartbeat timestamp of each node in Data Plane. In the background, the Reaper runs a thread that checks if a node has sent a heartbeat within the configured grace period. If it hasn't, the

Reaper invokes a callback to the Control Plane, triggering a topology rebalance. The details of Reaper implementation will be described in chapter 5.

Having established how the Control Plane detects failures and maintains an accurate view of which nodes are alive, the next important question is: how is data actually distributed across these nodes? Understanding the data distribution strategy is essential before discussing how the system rebalances data when failures occur.

3.3.3 Sharding strategy: Fixed Shard Model

As described previously, the system employs a fixed shard model, inspired by Redis Cluster. The keyspace is prepartitioned into a fixed number of shards that is a configurable value. Each key is deterministically mapped to a shard using hashing.

This approach decouples data distribution from the number of physical nodes. There can be 1024 shards defined, but only 5 nodes present, where each node would handle around 200 shards. When a new node joins, it will take ownership of some shard IDs without requirement of rehashing all existing keys.

Why Fixed Shards? This design provides:

- **Scalability:** Adding or removing nodes only affects the shard to node mapping, not the key to shard mapping.
- **Determinism:** Any client can independently calculate which shard holds a given key.
- **Flexibility:** The number of shards can be configured independently of the initial cluster size.

However system has to allow scaling, but it also has to do something on failures, so the process of rebalancing is introduced.

3.3.4 Shard Rebalancing Strategy

When nodes fail or join the cluster, the Control Plane must automatically reassign shards to maintain data availability and redundancy. This process is called **rebalancing**.

Handling Node Failures: When a node fails, the system performs an emergency promotion strategy:

1. Node failure mechanism detects the failure and notifies the Control Plane

2. For each shard where the failed node was primary, a replica is immediately promoted to become the new primary - this is almost instantaneous and decreases the unavailability of the system
3. The failed node is removed from the active node list
4. The system identifies shards that now lack sufficient replicas and assigns new nodes as backups

This ensures that data remains accessible even during failures, as long as at least one replica survives. Note that if there is no replica and the primary server dies, the data for these shards is lost - this system does not guarantee data durability. However, for the data to not be lost, at least one replica has to be alive. Users can add as many replicas as needed in configuration file, but each replica decreases performance of the system, as the primary node has to send data to a higher amount of servers.

Handling Node Additions (Lazy Assignment): When a new node joins:

1. The node is automatically assigned as primary for shards that have no current primary
2. It is assigned as a replica for shards that lack sufficient backups
3. Data is migrated to the new node using a synchronization protocol (described below)

This strategy is called "lazy" because it does the minimum necessary work. If a shard already has a healthy primary and the correct number of replicas, the system leaves it alone, it does not move or reassign it. The system only takes action for shards that are missing a primary or need more replicas. This approach reduces unnecessary data transfers and keeps the cluster stable during scaling.

Next chapter will describe how data is migrated between primary nodes and replicas.

3.3.5 Data Migration and Synchronization

Moving data between nodes while maintaining strong consistency and zero downtime requires careful coordination. The system uses a **four-phase synchronization protocol**:

1. **Bulk Copy:** The target node copies all data from the source node. Writes continue to be accepted by the source during this phase.
2. **Lock for Consistency:** The shard is temporarily locked to prevent new writes. This ensures a consistent snapshot can be taken.

3. **Incremental Catch-Up:** Any data written during Phase 1 is transferred to ensure the target has a complete, up to date copy.
4. **Unlock and Activate:** The shard is unlocked, and the target node is officially added as a replica or promoted to primary.

This protocol ensures:

- **Zero Data Loss:** All data is transferred before the migration is considered complete
- **Minimal Downtime:** Only Phase 2 blocks writes, and this phase is very short (just the incremental delta)
- **Consistency:** The lock ensures the target has an exact copy without missing any recent writes

The migration uses a **pull-based approach**, where the target node requests data from the source. This allows the target to control the transfer rate and retry on failures, making the protocol more resilient than a push-based approach where the source must track progress.

The system has to be resilient for network failures that cause cluster divisions. This problem was previously described and is known as Split-Brain problem. Below section describes how does the system handle this problem.

3.3.6 Split-Brain Prevention: Lease-Based Fencing

The Control Plane uses Raft consensus to prevent split-brain scenarios, but the Data Plane does not. Without additional mechanisms, a Data Plane node that loses network connectivity to the Control Plane might continue serving requests as if it were still the primary, leading to inconsistent data if a replica has been promoted in its place.

To prevent this, the system employs a lease-based fencing mechanism:

How Leases Work:

- Each Data Plane node periodically sends heartbeats to the Control Plane
- The Control Plane responds with an acknowledgment, granting the node a time limited validity lease (e.g., valid for 30 seconds)
- The node tracks its lease expiration locally
- Before processing any request (GET, PUT, DELETE), the node verifies that its lease has not expired

- If the lease has expired, the node immediately stops serving requests and returns an error (HTTP 503 Service Unavailable)

This is a form of self-fencing: nodes fence themselves out of the cluster if they cannot maintain connectivity to the Control Plane. This prevents a partitioned node from making conflicting writes.

Why This Prevents Split-Brain:

1. A network partition occurs, isolating Node A from the Control Plane
2. Node A can no longer send heartbeats or receive lease renewals
3. After the lease expires (e.g., 30 seconds), Node A voluntarily stops serving requests
4. Meanwhile, the Control Plane detects the failure via Reaper and promotes a replica to primary
5. The new primary begins serving requests with a valid lease
6. There is no window where both nodes believe they are primary and serve conflicting writes

This approach trades availability for consistency: a partitioned node becomes unavailable, but the system never produces inconsistent results.

The last part of the system is how the user interacts with it - the client.

3.4 Smart Client Design

Upon initialization, the client connects to the Control Plane to fetch the topology, which it stores in a local memory. For each request, the client hashes the key to a shard ID and looks up the primary node from the cached topology. The request is then sent.

3.4.1 Epoch-Based Request Validation

To prevent stale writes, every client request includes the client's current Epoch number (obtained from the topology). The Data Plane node compares the client's Epoch to its own:

- If the client's Epoch is older, the request is rejected with a specific error that signals the client to refetch the topology
- If the Epochs match, the request is processed normally

This prevents a client with outdated topology information from writing to a node that is no longer the primary, which could cause data inconsistency.

3.4.2 Adaptive Retry Strategy

The client implements an intelligent retry mechanism that adapts to different failure scenarios. On RPC failure, the client employs a tiered wait strategy:

- **Topology Mismatch:** If the node rejects the request because it's not the primary, the client immediately refetches the topology and retries (no delay)
- **Shard Locked:** If a shard is undergoing migration, the client waits briefly (e.g., 100ms) before retrying, allowing the migration to complete - as described previously, the shard is locked for a very short time, so this won't be an performance issue
- **Node Failure:** If a node is unreachable or fenced, the client waits longer (e.g., 200ms) to allow the Control Plane's Reaper (node failure detection mechanism) to detect the failure and promote a replica

However, as previously mentioned, the primary node can fail, but the request to replicas could succeed. Next section describes the idea of dealing with such an edge-case.

3.4.3 Client-Side Idempotency

To handle the case where a write succeeds on the server but the acknowledgment is lost (e.g., due to network issues or node failure), the client includes a unique idempotency token with each write or delete operation. This token is a UUID (randomly generated string with low probability of colliding that server as an id) that uniquely identifies the request.

When the client retries a request, it includes the same token. The Data Plane (and its replicas) track which tokens have been processed. If a retry arrives with a token that has already been processed, the server recognizes it as a duplicate and returns success without re-executing the operation.

Why This Matters:

1. Client sends PUT(key=X, value=Y, token=T) to Primary
2. Primary replicates to replicas and all nodes save token T
3. Primary crashes before sending acknowledgment to client
4. Client times out and retries the same request (same token T)

5. The promoted replica (now primary) recognizes token T and returns success
6. The operation is not duplicated, maintaining consistency

This mechanism ensures that retries are safe and that the strong consistency guarantee is maintained even when failures occur during write operations.

Chapter 4

Implementation of the Raft Consensus Module (pkg/raft)

The core of the distributed system is the Raft consensus algorithm, located in the `pkg/raft` package. The Raft implementation is responsible for managing the state of distributed nodes, ensuring data consistency across the cluster, and handling leader election and failure recovery. While the Data Plane uses synchronous replication to ensure data durability and strong consistency across replicas, the Control Plane requires the more sophisticated Raft consensus algorithm to manage the cluster's critical metadata with formal correctness and safety guarantees.

The implementation is highly modular, decomposing (breaking down) the Raft protocol into several distinct components: **Elector**, **Replicator**, **DataSaver**, and **Snapshotter**. Each of these components will be thoroughly described later. This separation of concerns allows for a clean concurrency model (a way to manage multiple tasks at once) where each component operates on its own set of goroutines, communicating via channels to minimize lock contention (a situation where many tasks wait for the same lock, slowing down the system).

Before diving into the specific components, it is important to understand the overall approach to concurrency and thread safety in this implementation.

4.1 Concurrency and Locking Model

Unlike simple implementations that wrap the entire state in a single mutex, the Raft implementation uses a `sync.RWMutex` (Read Write Lock) (a special lock that allows many readers but only one writer) for the core Raft state, while sub-components often use their own internal synchronization. A critical design choice is that the mutex is explicitly unlocked during `deliverToApplication` (when passing committed log entries to the application's state machine) to prevent deadlocks and to allow the

application to process data without blocking Raft’s consensus operations.

In chapter 3 I mentioned that gRPC integration with Raft is important. Below is a description of said interaction.

4.1.1 gRPC Connection Management

Maintaining stable gRPC connections in a dynamic cluster is handled by the `ConnectionManager`. This component must be configured with parameters such as initial backoff duration, maximum backoff duration, and health check intervals—these values are provided through environment variables when starting the system (showcased in Appendix).

- **Lazy Initialization:** Connections to peers are only established when needed (e.g., during the first replication attempt or vote request).
- **Exponential Backoff with Jitter:** If a connection fails, the manager uses a backoff strategy (waiting longer and longer between retries, starting at 100 ms up to 5 s) with added randomization (jitter). This prevents the “Thundering Herd” problem where all nodes try to reconnect to a recovering leader at the exact same moment, potentially overwhelming its network capacity.
- **Health Monitoring:** The manager periodically verifies the state of idle connections, closing and reopening them if they become stale or if an underlying network partition is detected.

All of the communication between servers is done with gRPC. Thanks to these mechanisms and the gRPC’s speed, the operations are both robust and performant.

With the concurrency model established and why gRPC is used, the following sections describe the individual components of the Raft implementation, starting with the node roles and lifecycle.

4.2 Raft Node Roles and Lifecycle

The system defines the node’s lifecycle through three primary roles: `Follower`, `Candidate`, and `Leader`. The transition between these roles is strictly controlled by the term logic and majority voting rules.

4.2.1 Implementation of Roles: Follower, Candidate, and Leader

The roles are defined as typed constants in `types.go`: `Follower`, `Candidate`, and `Leader`.

- **Follower:** The default state upon initialization (in `NewRaft`). Followers passively respond to RPCs from leaders and candidates.
- **Candidate:** A node transitions to this state when an election timeout occurs. This logic is contained in the `startElection` method in `election.go`.
- **Leader:** The node that handles all client requests and coordinates log replication. Transition to this role is handled by the `becomeLeader` function in `utils.go`, which initializes the `Replicator` instances for all peers.

As previously mentioned, the leader has its own election mechanism.

4.2.2 Leader Election Mechanism (`election.go`)

The `Elector` manages the node's election timer using a randomized timeout strategy (400–800 ms) to reduce the probability of split votes in large clusters.

1. **Timer Loop:** A dedicated goroutine (one that does only this task) runs `electionTimerLoop`, which listens on a `resetTimerCh`. This channel allows other components (like heartbeats or valid `AppendEntries` RPCs) to reset the timer efficiently.
2. **Safety Checks:** Before starting an election, the node verifies it is not currently the leader and is not in the middle of installing a snapshot, as snapshot installation is a heavy I/O operation that might temporarily delay heartbeat processing.
3. **Parallel Requests:** When a node becomes a Candidate, it broadcasts `VoteRequest` RPCs to all peers in parallel using individual goroutines, ensuring the election process is as fast as the fastest majority of the network.

With the leader elected, the data must be distributed to the other servers, log replication does that.

4.3 Log Replication Logic (`replicator.go`)

The replication logic is implemented using an isolated, per-peer model. This means that for every follower in the cluster, the leader spawns a dedicated `Replicator` goroutine that manages communication with only that specific follower. Each replicator operates independently, tracking its own state (like which log entries the follower has received) and sending `AppendEntries` RPCs at its own pace. This isolation ensures that if one follower is slow or unresponsive, it does not block replication to other followers. This section describes the architecture of the replicator, how log entries are prepared and sent, and finally how they are committed and applied to the state machine.

4.3.1 Isolated Replicators and the Pulse Protocol

Each `Replicator` maintains its own state for a specific peer, including `nextIndex` and `matchIndex`. This provides several advantages:

- **Fault Isolation:** A slow or disconnected follower does not block replication to other healthy nodes.
- **Instant vs. Periodic Replication:** The replicator waits on a `signalCh`. When the leader receives a new client request, it "pulses" this channel to trigger immediate replication. If no writes occur, a 50 ms ticker (repeating timer) ensures periodic heartbeats are sent to maintain leadership.
- **Self-Healing Quorum:** If a replicator detects too many consecutive failures (defined in `maxConsecutiveFailures` config variable), it triggers a `checkQuorumHealth()` call, which can cause a leader to step down if it realizes it no longer has a healthy majority.

With this architecture in place, the next question is how the leader prepares and sends log entries to its followers.

4.3.2 The Log Request (`AppendEntries`)

The protocol uses a sophisticated `prepareLogRequestArgs` method to calculate the exact suffix (end portion) of the log that needs to be sent. It handles the transition between sending incremental log entries (only the new entries since last update) and triggering a full snapshot installation if the follower's lag exceeds the available logs in memory.

Replication is triggered via the `LogRequest` RPC (standard Raft *AppendEntries*). The `Replicator` runs a continuous loop that monitors a signal channel. When triggered, it calculates the `prevLogIndex` and `prevLogTerm` to ensure log consistency. These arguments are packaged into a `LogRequestArgs` Protocol Buffer message and sent via the `PeerClient` interface.

Once followers acknowledge that they have received and stored the log entries, the leader can safely commit them.

4.3.3 Committing Entries (Commit Index) and Applying to the State Machine

The leader tracks the `matchIndex` (represented as `ackedLengths`) for each follower. When a majority of nodes acknowledge a log entry, the leader advances its `committedLength`. This logic is implemented in the `commitLogEntries` function in `raft.go`.

Upon commitment, messages are applied to the local state machine via the `Application` interface (defined in `application.go`). The application executes the command (e.g., updating the in-memory cache) and returns the result, which is then sent back to the client via a response channel.

As previously mentioned, the Raft saves some data to persistent storage. Next chapter describes it in more detail.

4.4 Asynchronous Persistence Management

To survive crashes and restarts, the Raft module implements strict persistence guarantees using the `DataSaver` struct located in `persistence.go`.

4.4.1 Writing Raft State to Disk (`persistence.go`)

The persistence layer uses Go's `encoding/gob` package (a specific binary format) to serialize data structures to disk. The `DataSaver` utilizes a worker pool pattern (a group of background tasks ready to handle work) to handle save requests asynchronously, preventing disk I/O (reading and writing to disk) from blocking the main Raft consensus loop.

The state is split into two files:

- **Metadata File:** Stores the `currentTerm`, `votedFor`, and `committedLength`.
- **Logs File:** Stores the append-only sequence of `LogEntry` objects.

4.4.2 State Recovery

Upon restart, the `LoadValues` method performs a sequential scan of the log file, reconstructing the in-memory log slice. Because logs are append-only (new data is only added to the end, never modified), the system handles partial writes by ignoring corrupted entries at the end of the file during recovery.

The system should support a very large amount of operations, but as the log grows the space needed to store them might be too big. Another problem caused by very big logs is how fast they are loaded on failure. Due to these problems, the snapshotting mechanism is implemented.

4.5 Log Compaction and Snapshotting

To prevent the log file from growing infinitely, the system implements a managed snapshotting mechanism.

4.5.1 Snapshot Trigger and Creation

When the log length exceeds the `snapshotThreshold`, the server initiates a snapshot. It is important to note, that due to the state being already synchronized between the servers, the decision to take a snapshot does not need to be agreed between nodes. Snapshot does not change the state, it is only an optimization of the space.

Snapshot requests a serialized state from the `Application`, writes it to a file, and then truncates (removes the old, already saved entries) the in-memory log.

4.5.2 Chunked Snapshot Transfer (`InstallSnapshot`)

Transferring a large snapshot over gRPC can be problematic as it might exceed the maximum message size or cause network timeouts. I implemented a `Chunked Transfer Protocol` in `sendInstallSnapshotRPC`:

- The snapshot file is read in 128 KB blocks.
- Each block is sent as a separate `InstallSnapshotRequest` containing an `offset` and a `Done` flag.
- The follower reconstructs the file using `WriteAt`, ensuring that the snapshot is durable (saved to disk and survives crashes) on the follower's disk even if the transfer is interrupted and resumed.

This mechanism allows the system to synchronize new nodes even when the dataset is significantly larger than available network buffers (temporary memory used to hold data being transmitted). Without it, the nodes that do not participate in snapshot transfer might see the leader as dead, as it wouldn't respond to the requests because of possible overhead from sending the entire snapshot.

Due to many possibilities of network speeds and memory size, the entire Raft algorithm needs to be highly configurable.

4.6 System Configuration and Environment Parameters

The Raft module is highly configurable through environment variables, allowing it to be tuned for different network conditions or testing scenarios.

Table 4.1: System Configuration Parameters

Variable	Default	Description
RAFT_ID	<i>required</i>	Unique integer ID for the Raft node.
TOTAL_NODES	<i>required</i>	The total number of nodes in the cluster.
RAFT_ADDRS	<i>required</i>	Comma-separated list of all node addresses.
SNAPSHOT_THRESHOLD	<i>required</i>	Number of log entries before a snapshot is triggered.
RAFT_INITIAL_BACKOFF	1 s	Initial delay for gRPC reconnection attempts.
RAFT_MAX_BACKOFF	30 s	Maximum delay for reconnection attempts.
RAFT_ARTIFICIAL_DELAY	0 ms	Simulates network/CPU lag for performance testing.

Chapter 5

Implementation of the Control Plane and the Data Plane

As introduced in Chapter 3, the system architecture is separated into two distinct layers: the **Control Plane** and the **Data Plane**:

- **Control Plane** - Manages cluster metadata using the Raft consensus algorithm. It tracks which data nodes are alive, which nodes hold which shards, and coordinates rebalancing when failures occur. The Control Plane does not store user data; it only manages the topology.
- **Data Plane** - Stores the actual key-value data. Data nodes handle client requests (GET, PUT, DELETE) and synchronously replicate data to their assigned replicas to ensure strong consistency.

This chapter presents the detailed implementation of both planes, starting with the Control Plane's topology management and failure detection, then moving to the Data Plane's storage, replication, and client interaction mechanisms.

5.1 Implementation of the Control Plane

The Control Plane node is the central orchestrator of the entire application. It is responsible for maintaining the system state and ensuring that all nodes remain synchronized. Controller is a singular node in the Control Plane. The main job of Control Plane is to manage system topology.

5.1.1 Topology State Machine

The core of the Control Plane is a Replicated State Machine (a system where every controller has its own identical copy of the state) responsible for managing cluster

metadata. This ensures that, in a multi-node setup, all controller instances agree on which data node holds which shard. To make sure that Control Plane's data is consistent and correct, the Raft algorithm is used.

The state represents:

- Epoch: A monotonically increasing number representing the version of the topology. This works similarly to a Raft term. It stores the information how many times the topology has changed due to node failures or addition or removal.
- Nodes: A map of all registered data nodes with information about their addresses and statuses.
- Shards: A map of all shard IDs to their primary and replica data node IDs.

The Control Plane implements the Raft application interface, processing commands such as node registration and topology updates. Changes to the topology are not applied until they are committed by the Raft algorithm to ensure strong consistency. When a command is sent to the controller, it updates its local configuration and, if the controller is the leader, it may trigger a rebalance. To prevent the Raft log from growing infinitely, the system supports snapshots, which save the current cluster configuration to durable storage.

Due to the Control Plane being built on top of the Raft algorithm, the system is fault tolerant to singular controller failures. However, the Control Plane has direct communication with the Data Plane, where the Raft algorithm is not implemented. Each data node might fail or lose access to other controllers, so monitoring whether they are healthy is crucial.

5.1.2 Node Health Monitoring

A dedicated component called the Reaper (as defined in the Terminology section at the beginning of chapter 2) is responsible for monitoring the system for failures. Data nodes are designed to periodically send heartbeats to the Control Plane, which updates the timestamp of the last received heartbeat for each node. The Reaper runs a background goroutine that wakes up every second to check the liveness of all data nodes. If a node has not sent a heartbeat within the configured time period, it is marked as dead. In this case, the Reaper triggers a mechanism that signals the Control Plane to take action. If the failed node was a primary node, one of its replicas is promoted to the primary role. Subsequently, the rebalancing logic will attempt to find a new node to act as a replica to restore the desired redundancy level.

Due to radical changes in the topology, the rebalancing mechanism must be introduced.

5.1.3 Shard Rebalancing

Rebalancing is responsible for ensuring that data availability and the even distribution of data are preserved.

When a node failure is detected, the system performs an emergency promotion:

1. The Controller identifies all shards where the failed node was the primary.
2. For each such shard, the first available replica is promoted to become the new primary. Since the original primary is dead, no data can be copied; the promoted replica relies on the data it already possesses.
3. The dead node is removed from the active node collection.

Note: When both the primary node and its replica die, the data in those shards is lost. This is a direct consequence of not providing a durable cache, as noted previously in introduction. While it is possible to increase the number of replicas, doing so will slow down the cache, as the system must ensure the primary node replicates data to every replica before returning an OK status to the client. However, it is very unlikely for both nodes to die simultaneously if they are located on different physical servers. The default number of replicas is set to 2, but due to system being highly customizable, the amount of replicas can be increased.

Rebalancing is triggered automatically when a new node registers or a failure occurs. It uses a greedy strategy:

1. It identifies all shards that lack a primary node and assigns them to ACTIVE nodes using a round-robin approach (a simple method of rotating through available servers one by one).
2. It ensures that every shard has the required number of replicas, selecting new nodes to act as replicas if needed.
3. For initializing these new replicas or during planned load balancing, a synchronization protocol is used to ensure data consistency.

To initialize new replicas, the Controller implements a four-phase synchronization protocol:

1. The target node is instructed to perform a bulk copy of all data from the current primary node.
2. The shard is marked as LOCKED in the cluster topology. This temporarily stops writes to this specific shard to ensure a consistent state.

3. A final incremental catch-up is performed to transfer any data that was added during the initial bulk copy in phase 1.
4. The target data node is officially added to the replica set, and the shard is set back to ACTIVE.

The Control Plane exposes HTTP endpoints that provide a simplified interface for external communication. These endpoints abstract the complexity and allow client and Data Plane to perform operations without needing to understand the mechanisms of the Control Plane.

5.1.4 Controller HTTP API Reference

The Control Plane exposes several internal and external HTTP endpoints. These are used by data nodes for registration and by the Smart Client for topology discovery.

Table 5.1: Controller HTTP API Endpoints

Endpoint	Method	Description
/register	POST	Registers a new data node with its address.
/heartbeat	POST	Receives health updates and returns the current Epoch.
/topology	GET	Returns the full JSON description (a standard text-based data format) of the cluster state.
/pull	POST	Triggers a shard migration from a source to a target node.

While emergency promotion handles sudden failures, the system also manages the lifecycle of nodes entering and leaving the cluster gracefully. The system allows to scale by adding new data nodes, the next chapter will describe it.

Scaling Up: The Lazy Assignment Strategy

The system uses a fixed shard model inspired by Redis Cluster, where the keyspace is pre-partitioned into a configurable number of shards (e.g., 10 or 1024). Each key is deterministically mapped to a shard using a hash function: $\text{shardID} = \text{hash}(\text{key}) \% \text{totalShards}$. This decouples data distribution from the number of physical nodes.

When a new data node registers, the `Rebalance()` function is invoked and follows a *Lazy Assignment* strategy:

- **Automatic Shard Assignment:** New nodes are automatically assigned as primaries for shards that do not yet have a primary. The system uses a round-robin approach (an approach in which the choice is circular - it goes in loop to choose a server) to distribute primaries evenly across all active nodes. For

example, if the cluster has 10 shards and 3 nodes join, Node 1 might get shards 0, 3, 6, 9; Node 2 gets shards 1, 4, 7; and Node 3 gets shards 2, 5, 8.

- **Replica Population:** After assigning primaries, the rebalancer identifies shards that lack the desired number of replicas and assigns new nodes as replicas.
- **Data Synchronization:** For each newly assigned replica, the system triggers a background data pull from the primary node to the replica using the `triggerPull` mechanism, ensuring the replica has a complete copy of the shard data.

Because shards are prepartitioned and the hash function is deterministic, adding a new node does not require rehashing all keys. The new node simply takes ownership of specific shard IDs, and the Smart Client automatically routes requests to the new primary once it fetches the updated topology. For instance, if a cluster grows from 2 to 3 nodes, roughly one-third of the shards (and thus one-third of the data) will be reassigned to the new node, achieving automatic load distribution.

For manual redistribution of already assigned shards (e.g., to further optimize load), administrators can use the `MoveShard` command, which utilizes a four phase synchronization protocol to migrate a shard from one primary to another with zero data loss and minimal downtime.

Similarly to data growing infinitely, sometimes the data might shrink to not need so many servers. Due to this reason, the system can also scale down by removing data nodes.

Scaling Down and Replica Recovery

The system treats a deliberate node removal similarly to a failure. If a node is decommissioned:

- **Redundancy Restoration:** If the removed node was a replica, the `Rebalance` logic detects that the shard's replication factor has decreased. It automatically identifies another healthy node in the cluster and triggers a data pull to restore the backup.
- **Safe Promotion:** If the node was a primary, promotion happens first, followed by replica restoration on a different node.

This ensures that the cluster is self healing regarding its durability guarantees; as long as there is a spare node in the cluster, the system will always attempt to maintain the configured number of replicas for every shard.

As described in Chapter 2, the common problem of distributed systems is the split-brain problem. It is characterized by nodes losing connection to each other and in result a cluster splitting from one into two or more.

5.1.5 Lease Management and Fencing

Lease management is the system's primary defense against the split-brain problem. Data nodes are implemented to send heartbeats to the Control Plane to signal that they are alive. If the Controller acknowledges the heartbeat, the data node extends its validity timestamp. A lease is a configurable duration during which a data node is considered valid. We use fencing (a technique that makes node physically or logically disable its peer to ensure it cannot write to shared storage) to verify the lease by comparing the current time to the last validity timestamp. If the difference exceeds the lease time, the data node immediately stops serving requests. To ensure this condition is always checked, every PUT, GET, and DELETE operation verifies the lease. If the lease has expired, the data node returns an HTTP 503 **Service Unavailable** response, effectively fencing the node out of the cluster.

This section described how the Control Plane works with the Data Plane. The next section will describe the details of Data Plane.

5.2 Data Plane

The Data Plane is responsible for storing the data and the retrieval of said data. This section describes the design and implementation of data node components.

5.2.1 Concurrent Data Store

The data storage system provides a simple but thread safe in memory key-value store. It uses a standard Go map structure wrapped with a mutex. The data store supports data migration and, in particular, implements two core functions:

1. `ExportShard(shardID, totalShards)` : A crucial function for rebalancing, it iterates through the entire map, hashes each key to determine its shard, and returns only those keys belonging to the given shard ID.
2. `Import(data)` : This function bulk-loads data into the map and is used during shard migration or recovery.

Next chapter will describe how the Data Plane manages synchronization with the Control Plane on its local servers.

5.2.2 Synchronization and Consistency

The Data Plane maintains consistency through coordination with the Control Plane.

The Data Plane ensures that the topology is always synchronized. Heartbeat responses contain the latest Epoch. If the Epoch is higher than the node's local Epoch value, the lease manager will immediately fetch the latest topology from the Control Plane and update the local state. This ensures that the data node is always aware of the role assigned to it by the Control Plane—whether it is a primary node or a replica.

The Data Plane also uses epoch-based fencing. If a node has an active lease, every write request must include an Epoch. The data node compares the client's Epoch to its own; if the request's Epoch is smaller than the local one, the request is rejected with a **412 Precondition Failed** status, preventing stale writes from delayed clients that have yet to refetch the current topology.

The final synchronization mechanism is primary-backup replication. If a data node is a primary, it is responsible for replicating writes to all replica nodes. This replication is synchronous (meaning the system waits for all replicas to finish before returning a success message to the user). This makes the system strongly consistent, as a request is only accepted if all replicas and the primary node have successfully saved it. This also provides a form of durability, as the system preserves data as long as at least one replica remains alive.

Note: The strong consistency definition in Chapter 2 states that:

- If operation A completes (the client receives an OK), then every operation B that starts after A finishes must see the effect of A.
- If an operation A is in-flight or timed out, it is allowed to either "have happened" or "not have happened," provided the system remains consistent about that choice thereafter.

If all replicas receive a write request but the primary node dies before acknowledging it, we encounter a state of uncertainty—the second scenario. However, since all replicas recorded the update, the client will see the request as accepted once the failover completes. In the case where only some replicas received the write before the primary crashed, the system might be inconsistent. This is resolved by the Smart Client via client-side idempotency (a mechanism that ensures an operation can be safely repeated without changing the result), which is discussed in more detail in Section 5.3.2.

As described in the Control Plane, the data might be sent between nodes in the Data Plane, so each server must support these operations.

5.2.3 Data Transfer Protocol

Shard synchronization is a crucial part of the system. Two data nodes - a source and a target—must synchronize their data. The source node exports its data, while the target node imports it. The Data Plane uses a pull model:

1. The Controller sends a pull request to the target node.
2. The target node sends a GET request to the source node's export endpoint, which returns the result of the `ExportShard()` function (see Section 5.2.1).
3. The target node calls the `Import()` function (see Section 5.2.1) using the data received from the source node.

Similarly to the Control Plane, to abstract the implementation of operations, the data node also exposes HTTP endpoints for its operations.

5.2.4 data node HTTP API Reference

Data Plane nodes expose endpoints for client CRUD operations and internal synchronization.

Table 5.2: Data Plane nodes HTTP API Endpoints

Endpoint	Method	Description
<code>/data</code>	GET	Retrieves a value by key if the node is the primary.
<code>/data</code>	POST/PUT	Stores a key-value pair and replicates to backups.
<code>/data</code>	DELETE	Removes a key and replicates the deletion.
<code>/replicate</code>	POST	Internal: Replica receives a data write from the primary.
<code>/delete-replicate</code>	POST	Internal: Replica receives a delete command from the primary.
<code>/export</code>	GET	Internal: Exports shard data for migration.
<code>/import</code>	POST	Internal: Imports shard data from a source node.

Shard Locking: During migration (Phase 2 of the four-phase protocol), the shard is set to `StatusLocked`. Any client request to a locked shard is rejected with an HTTP 423 `Locked` status, signaling the Smart Client to wait and retry.

While the Control Plane handles high level cluster orchestration and the Data Plane handles lower level data logic, the Client is responsible for efficient operations and ensuring reliable communication. Next section will describe the client.

5.3 Smart Client Mechanism

The Smart Client is an implementation that eliminates the need for a centrally implemented load balancer or proxy, which can increase latency or compromise data consistency (discussed in more detail in Chapter 3).

The client maintains a local copy of the cluster topology fetched from the Control Plane. At the start of a session, the client queries one of the controllers to obtain the latest topology. It must hold its own Epoch to include in every request and verify whether its topology state is current. The client also tracks the list of active data nodes and shard-to-node mappings to route requests correctly based on the key's hash.

Every GET, PUT, and DELETE operation is associated with a key. The client calculates the hash of the key to route the request to the correct node:

1. Hash the key using the FNV-1a algorithm (see Section 2.7.1).
2. Determine the shard ID by taking the hash modulo the number of shards.
3. Look up the primary data node ID for that shard in the local cache.
4. Send the request to the identified primary data node.

The client is lazy but reactive, continuing to use its local cache as long as it remains valid. Cache validity is determined by data node responses:

1. **412 Precondition Failed:** The client's Epoch is too old; the data node has a newer Epoch.
2. **400 Not Primary:** The shard has migrated or a failover has occurred.

In both cases, the client must refetch the topology and associated metadata from the Control Plane (as described in Section 3.4) and update its local cache.

However, due to operating over the network with many possible points of failures, the client must be resilient to them. Next section will describe how the client manages problems.

5.3.1 Adaptive Retry and Backoff Strategy

To ensure resilience against transient failures, the Smart Client implements an adaptive retry loop:

- **Retry Limit:** Each operation is attempted up to `maxRetries` (default: 5) times.
- **Role Miss Retry:** If a node returns `400 Not Primary`, the client immediately refetches the topology and retries the request on the new primary.
- **Migration Wait:** If a shard is locked (`423 Locked`), the client waits for 100 ms before retrying, allowing the migration protocol to complete Phase 2.
- **Failover Wait:** If a node is fenced or unreachable (`503 Service Unavailable`), the client waits for 200 ms to allow the Control Plane's Reaper to detect the failure and trigger an emergency promotion.

This tiered wait strategy balances system responsiveness with the need to avoid overwhelming a cluster that is undergoing reconfiguration.

There can be a case, where the operation succeeded, but the response was not correctly sent back to the client.

5.3.2 Client-Side Idempotency

To prevent duplicate operations during retries (especially when a primary node has died), the client includes an idempotency token and a client ID in each request. Every write and delete operation uses a unique UUID (a randomly generated string with very low probability of colliding) token, which is used to track the status of the operation. Each log entry in the Raft algorithm is designed to hold the idempotency token and client ID. If the client is unsure whether an operation succeeded, they can safely resend the request; if it has already succeeded, the system will identify the previous request and ensure no duplicate processing occurs. If it failed, the system will retry it.

This mechanism ensures that the primary node in the Data Plane acts as the coordinator for the token. When writes are replicated, the token is passed along to the replicas. If the primary node fails after replicating but before acknowledging the client, the promoted backup already possesses the request and its token. If the client then resends the request, the new primary will recognize the token and reply with an OK status. This solves the previously discussed problem where only some replicas receive a write before a primary crash (see Section 5.2.2).

Consider a situation with a primary node P and two replicas R_1 and R_2 . The client sends a request `PUT(key, value)` with token T . Primary node P replicates to R_1 , which saves the request, but P crashes before replicating to R_2 or acknowledging the client. The client request times out, and the client resends it. We have two possibilities:

1. R_1 becomes the new primary: It recognizes the request with token T and returns OK. Through shard synchronization (see Section 5.2.3), R_2 will eventually receive this update, maintaining a consistent state.
2. R_2 becomes the new primary: Since R_2 did not receive the initial request, it will process the resent request, write it to memory, and replicate it to R_1 (overwriting any partial state). The system remains consistent.

This logic extends naturally to any number of replicas.

5.4 Control Plane API Layer (pkg/api)

The `pkg/api` package provides the external interface for the Control Plane. While internal cluster coordination uses gRPC, administrative tasks and Control Plane state queries are exposed via a traditional HTTP/JSON API.

5.4.1 Bridging Raft and HTTP

The `api.Server` acts as a wrapper around the Raft node. When an HTTP request arrives (e.g., to update a configuration), the server must ensure the request is handled by the current leader.

- **Leader Caching:** To avoid the latency of checking leadership status on every request, the API server maintains a `LeaderCache`. It caches the address of the known leader for a short period (1 s), refreshing it only if a request to that leader fails.
- **Command Forwarding:** If a client hits a follower node with a write request, the follower uses the gRPC `Forward` method to send the command to the leader. The leader then proposes the command to the Raft log.

5.4.2 Idempotency and Resilience

Distributed consistency requires handling retries safely. The API layer implements two critical sub-components:

1. **Idempotency Cache:** To prevent the same administrative command from being executed twice (for example, registering the same data node multiple times due to a network timeout), the server tracks request IDs in an `IdempotencyCache`. It discards duplicate requests before they reach the Raft log.
2. **Auto-Retriever:** Transient failures are handled by a `Retriever` component, which automatically retries idempotent operations if the connection to the leader is interrupted, ensuring high availability for Control Plane operations.

5.4.3 Client API Reference

The following table lists the endpoints exposed by the API server for external interaction:

Table 5.3: Client HTTP API

Endpoint	Method	Description
<code>/kv</code>	POST	Proposes a generic KV update to the Raft state machine.
<code>/kv/{key}</code>	GET	Reads a value from the replicated state machine.
<code>/kv/{key}</code>	DELETE	Deletes a value from the state machine.
<code>/health</code>	GET	Returns the liveness status of the Controller node.
<code>/status</code>	GET	Returns technical Raft details (Term, CommitIndex, Role).
<code>/leader</code>	GET	Returns the address of the current cluster leader.

Chapter 6

Correctness verification and Fault injection testing

6.1 Methodology for testing distributed systems

As testing distributed systems requires independent servers I considered two options. First option was to run tests using multiple computers. This option however wasn't developer friendly. Every time I would want to test my system I had to boot multiple computers, synchronize data through version control system, then set it up. This would take too much time and effort just to run some basic testing. That's where the second option came in - I simulated distributed servers using Docker Containers [16] (a way to package and run software in an isolated environment). As I can control almost everything about Docker Containers - restarts, failures, networks, this was my way of testing.

To facilitate consistent and repeatable testing, the entire system is orchestrated using `docker-compose`. The current setup simulates a cluster composed of 3 Controllers and 5 Data Nodes. This configuration is defined in the `docker-compose.yml` file, which manages:

- **Network Isolation:** It defines two distinct bridge networks: `raft-cluster` for internal gRPC communication between Raft nodes, and `client-access` for external API requests.
- **Persistence:** Volumes are mapped for each controller to ensure that Raft logs and snapshots survive container restarts, allowing for testing of recovery mechanisms.
- **Service Dependencies:** In Docker Compose, dependencies define the order in which containers start. Data nodes are configured to depend on controllers, meaning Docker Compose will start the controllers first and wait for them to be ready before starting the data nodes. This mirrors the real-world startup

sequence where data nodes must register with a functional Control Plane—if a data node starts before any controllers are available, it would have nowhere to register and would fail to join the cluster.

This containerized approach allows for complex failure scenarios (like killing a node or partitioning the network via `docker network disconnect`) to be automated in scripts.

6.2 Unit and integration tests for the Raft implementation

As noted in chapter 3, Raft is a central thing in Control Plane, and Control Plane is base for everything in the system. Because of that my Raft algorithm implementation had to be thoroughly tested. I implemented unit tests for every major method in the system with edge cases. In particular I have unit tests of:

- elections - basic check whether it works on 3 and 5 nodes, then tests around replying to various Term value, then Election timeout test
- messages - creation, conversion between messages struct and grpc definitions
- persistence - saving logs, loading logs, comparing whether loaded logs are correct, saving empty logs, saving to invalid paths, saving to corrupted file
- snapshot - whether indexes in log are correct after snapshot, whether snapshot is saved correctly - after load to we have correct data, checking whether log is truncated after snapshot, test for snapshot installation rpc, checking whether snapshot will run if below threshold or log empty, checking whether snapshot creates with any directory problems

But as far as unit tests (tests that check a single small part of the code in isolation) can go, they cannot test how parts of the system behaves in terms of each other. There come integration tests (tests that check if different parts of the system work together correctly):

- Full test of election, writing to a log and replication of this log to other nodes
- Test of forwarding to a leader if broadcast is sent to non-leader node
- Test of Raft recovery - whether node will have correct values after it crashes

There was also a benchmark (a test used to measure the performance or speed of a system) created for Raft only. It tried to run as many requests per second as possible. On Macbook Air M1 2020, 16GB RAM it achieved around 290k requests per second.

6.3 Simulation of failures and network partitioning

While the Raft tests are useful for testing Raft, they are not useful for testing the entire system. System requirement was to be Strongly Consistent and have Partition Tolerance. To verify these properties, I developed a suite of automated shell scripts located in `tests/fault_injection/` and `tests/e2e/`.

The two primary verification scripts are:

1. `test_network_partition.sh`: This script verifies Partition Tolerance by executing the following sequence:
 - Start the system and wait for it to reach a stable state.
 - Identifies the current Raft leader and disconnects it from the network.
 - Waits for the remaining nodes to detect the failure and elect a new leader.
 - Modifies the cluster topology (e.g., adding a node) via the new leader.
 - Reconnects the original leader and verifies that it successfully synchronizes with the new majority and reflects the changes.
2. `test_strong_consistency.sh`: This script verifies the system's linearizability (strong consistency) by performing multiple sub-tests:
 - First configured some topology of the system to look like it was running for some time to check read-after-write on it. The test run PUT request for some random value to the key and without waiting asked for the value. The expected value of read was the write value.
 - Second was to write sequence of values to the same key (change the key value multiple times). Then there was one singular read sent. The expected value was the value of the last write.
 - Third part was similar to first, but this time there were multiple keys used.
 - Fourth part was about Control Plane consistency, the test tried to register a new node and expected the registration to succeed and be visible in following read
 - Fifth part was to expect all Raft nodes to be available after all of these operations from previous subtests
 - Sixth part was to write a value to some key and restart data-node holding it. The expected outcome was to data-node to have this value in the memory.
 - Seventh part was a rerun of second part, to make sure that data-node after restart is still functioning in the intended way.
 - Eight, final part was to write a value to data-node and then immediately do a read-after-write check whether replica has this data.

3. **basic_e2e_test.sh**: Located in the `tests/e2e/` directory, this script performs a full lifecycle test: starting the cluster, registering nodes, storing a large set of keys, and verifying they can be retrieved after a series of controlled node restarts.

All of these tests succeeded, guaranteeing that the system is Strongly Consistent and Partition Tolerant.

6.4 Continuous Integration (CI/CD)

To maintain code quality and prevent regressions (new bugs in old code) during development, I implemented a Continuous Integration (CI) pipeline—an automated process that builds and tests the code every time a change is made—using **GitHub Actions**. The workflow is defined in `.github/workflows/go.yml` and performs the following steps on every pull request or push to the main branch:

- **Environment Provisioning**: It sets up a Linux environment with the required Go version (currently 1.24).
- **Dependency Caching**: To optimize execution time, the pipeline caches Go modules and build artifacts, typically reducing the build stage duration by over 50%.
- **Automated Testing**: It executes the command `go test ./... --short`. The `--short` flag is used in the CI environment to skip long-running benchmarks or extremely heavy integration tests while still verifying the correctness of all core logic, including Raft state transitions and data node CRUD operations.

This automated feedback loop was essential for the rapid development of complex consensus logic, where a small change in term handling could otherwise have catastrophic effects on cluster stability.

Chapter 7

Performance Analysis and Comparison with Existing Solutions

7.1 Benchmarks

The primary advantage of a cache is its speed. If performance were not a priority, a traditional database would be a more suitable choice. Consequently, a thorough performance analysis is crucial.

I conducted benchmarks to calculate the Requests Per Second (RPS) (the throughput or speed) the cache can support. Each benchmark was executed for 10 seconds using 100 workers (clients) implemented as threads. Unless specified otherwise in test, all tests will use 100 workers. The tool issues as many write requests as possible, with each request targeting a unique key. After 10 seconds, the total number of requests is divided by 10 to determine the average RPS.

The benchmarks were performed on a MacBook Air M1 (2020) with 16 GB of RAM, running macOS Tahoe 15.1.

To ensure objective and reproducible results, I developed a suite of benchmarking tools located in the `tests/benchmark/` directory:

- `benchmark_rps.go`: A high-performance Go based benchmark that measures maximum RPS using goroutines and connection pooling.
- `benchmark_scaling.sh`: Orchestrates tests across multiple cluster configurations to measure horizontal scaling (adding more servers to increase performance) efficiency.
- `benchmark_raft_delay.sh`: Evaluates the impact of consensus latency on

Control Plane performance by injecting artificial Raft processing delays.

- `benchmark_comparison.sh`: Automates the deployment of both this system and competitor systems (Etcd, Redis) to provide direct side-by-side performance comparisons.

Default Cluster Configuration: Unless specified otherwise, all benchmarks in this chapter will use the default configuration of 3 controllers and 5 data nodes. This configuration provides a realistic deployment scenario, that balances fault tolerance on the Control Plane with sufficient data distribution. Some benchmark sections below will override this configuration to test specific scaling scenarios, but it will be noted what is changed.

7.2 Throughput Study (RPS) Depending on the Number of Clients and Shards

Table 7.1: Client Scaling (One data node)

Workers	RPS
1	5,714
10	18,855
50	28,336
100	29,605
200	27,661

Table 7.2: Horizontal Scaling (Multiple data nodes, No Replication)

data nodes	Combined RPS	Scaling
1	28,460	1.0x
2	55,001	1.93x
3	72,731	2.56x

Table 7.3: Replication Impact (Per-node Throughput)

Configuration	RPS
1 data node (no replication)	28,909
2 data nodes (no replication)	27,100
3 data nodes (no replication)	27,468
2 data nodes (WITH replication)	26,108
3 data nodes (WITH replication)	27,657

7.3 Analysis of Raft Protocol Overhead on Latency

Latency is the delay between a user's request and the server's response. I created a benchmark for a case where Raft was used for every single operation. In a standard deployment, this is not necessary because the Smart Client hashes the topology and does not need to consult the Control Plane for every request, but only when failures occur.

However, in this specific benchmark, I forced the client to send every request through the Raft consensus module. This resulted in approximately 8,000 requests per second. I then implemented an artificial delay for the broadcast and *LogRequest* RPC operations, allowing the system to simulate varying network or processing latencies. This demonstrates the extent to which Raft performance influences the overall system. The results were as follows:

Table 7.4: Analysis of Raft Protocol Overhead on Response Time

Artificial Delay	Consensus RPS	Impact
0 ms	8002	Baseline
10 ms	274	29x slower
25 ms	100	80x slower
50 ms	59	135x slower

This clearly illustrates the significant impact that Raft consensus speed has on the overall system performance.

7.4 Comparative Study

To evaluate the performance of my implementation, it is necessary to compare it against established solutions. I identified two primary open-source systems for comparison, based on current industry popularity [7]. The first is Etcd, which is also a Raft based distributed key-value store that guarantees strong consistency. However, Etcd is a durable store, meaning data is persisted to disk to survive node crashes. The second system is Redis, the most widely used cache in the world [8, 7]. Redis is primarily an AP (Availability and Partition Tolerance) system, meaning it prioritizes performance over strong consistency, offering eventual consistency instead (for more information, read chapter 2 again). Redis is a highly optimized system written in C with over 15 years of development. Matching the performance of Redis is a significant challenge, making it an excellent benchmark for speed.

Both Etcd and Redis use default Docker containers created by their development teams. Redis uses container `redis:7-alpine` with default options and Etcd uses `gcr.io/etcd-development/etcd:v3.5.11` with default options.

7.4.1 Performance Comparison with Etcd

Etcd was deployed using Docker. I utilized the official client package `go.etcd.io/etcd/client/v3@v3.5.11`.

I performed ten iterations of the 10-second, 100-worker benchmark described in Section 7.1. The results were averaged, yielding a score of 26,661 RPS.

7.4.2 Performance Comparison with Redis

Redis was also deployed using Docker, and the official `github.com/redis/go-redis/v9` client was used. Following the same methodology as with Etcd, the averaged result over ten runs was 38,776 RPS.

7.4.3 Conclusions from the Comparative Analysis

I re-ran my system alongside Etcd and Redis using the same 10x10-second, 100-worker configuration. My implementation averaged 27,715 RPS, making it approximately 3.95% faster than Etcd and 30% slower than Redis.

These results are consistent with expectations. My system is faster than Etcd, likely due to the lack of disk persistence in the Data Plane, but slower than Redis due to highly optimized C implementation, more lenient consistency guarantees and extensive optimizations done during its many years of functioning. Further optimizations to the Raft implementation could close the gap, but reaching Redis-level performance would be extremely difficult given the architectural trade-offs required for strong consistency and the limitation of Go programming language.

Chapter 8

Summary

8.1 Evaluation of the Achievement of Research Objectives

The functional and non-functional requirements of the system have all been met. Every requirement was verified with a corresponding test case to ensure correctness.

The most significant challenge of the project was the implementation itself. Since the original Raft paper provides high-level algorithmic details rather than low-level implementation specifications, designing the concurrency model and operational logic was my primary responsibility.

Initially, I implemented Raft using multiple mutexes, which made the system difficult to reason about and led to several deadlocks that required extensive debugging. Another challenge was the delayed implementation of snapshotting logic. This required a significant refactoring of the log indexing and term management to correctly handle truncated logs.

Furthermore, the initial implementation of snapshots was inefficient. Sending a large snapshot in a single request blocked the main thread and held the state mutex for too long, making the leader unresponsive to heartbeats. This caused a bug where another node would detect a leader failure and initiate a new election during the transfer. To resolve this, I implemented a chunked transfer protocol, which allowed the leader to remain responsive while synchronizing followers.

8.2 Practical use of the software

It is important to note the practical limitations of such software as distributed cache with strong consistency. This thesis presents that it is possible to write such a cache, but it does not mean that it is always a good idea to use it. Most cloud providers offer flexible RAM memory options up to multiple terabytes. This amount of RAM

in cache will be overkill for even most complex web applications or e-commerce platforms. A single node cache like Redis will be more than sufficient. A single Redis instance can handle hundreds of thousands of requests per second with very minimal latency, which satisfies the needs of most applications. However, there are some scenarios where distributed cache might be useful or even necessary:

- **Hyperscale Applications:** Social media platforms like Facebook, Twitter or some streaming services like Youtube or Netflix with hundreds of millions of active users might have to store too much data to fit into even the most powerful machines cloud operators have
- **Geographic Distribution:** Some applications might serve users across the globe and having only one server might create latency so big, that it might be beneficial to add multiple servers in many regions of the globe to minimize the network latency.
- **Regulatory Compliance:** Some industries require some part of the data to be stored within specific geographic boundaries.

But for the most developers and companies, the choice of a simple single node cache will be almost always sufficient. Distributed cache mostly excels in extremely big scale or when geographics come into consideration.

8.3 Directions for Further Development

As discussed in Chapter 7, the performance of the Raft consensus module is the primary bottleneck for the entire application. I have identified several potential areas for further improvement:

- **Batching Requests:** Currently, log entries are persisted for every request. Performance could be significantly improved by batching multiple entries into a single I/O operation. This would reduce the frequency of file open and close operations. However, this must be implemented carefully; if the system acknowledges a request before the batch is safely persisted to stable storage, it would violate strong consistency guarantees.
- **I/O Optimization:** Disk I/O is a slow operation. The system could benefit from keeping log files open based on a heuristic to avoid repeated overhead. Frequent `fsync()` calls are necessary for reliability, but a more sophisticated file management strategy could bridge the performance gap without jeopardizing log persistence.
- **Encoding Performance:** Profiling reveals that binary encoding and decoding account for approximately 40% of the application's processing time. Imple-

menting a custom, more efficient serialization format could significantly reduce this overhead.

- **Concurrency Tuning:** The system might benefit from a more optimized thread-to-core mapping for handle-intensive I/O operations.

Bibliography

- [1] E. Brewer, "CAP twelve years later: How the "rules" have changed" in *Computer*, vol. 45, no. 2, Feb. 2012.
- [2] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm" in *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, Philadelphia, PA, 2014.
- [3] T. J. Edwards et al., "CockroachDB: The Resilient Geo-Distributed SQL Database" in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020.
- [4] S. Zhou and S. Mu, "Fault-Tolerant Replication with Pull-Based Consensus in MongoDB" in *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, 2021. [Online]. Available: <https://www.usenix.org/conference/nsdi21/presentation/zhou>. [Accessed: 21-Dec-2025].
- [5] Neo4j, "Causal Clustering Architecture" [Online]. Available: <https://neo4j.com/docs/operations-manual/current/clustering/introduction/>. [Accessed: 20-Dec-2025].
- [6] RabbitMQ, "Quorum Queues" [Online]. Available: <https://www.rabbitmq.com/docs/quorum-queues>. [Accessed: 20-Dec-2025].
- [7] DB-Engines, "Ranking of Key-Value Stores" [Online]. Available: <https://db-engines.com/en/ranking/key-value+store>. [Accessed: 20-Dec-2025].
- [8] Redis, "Get Started with Redis" [Online]. Available: <https://redis.io/docs/latest/get-started/>. [Accessed: 21-Dec-2025].
- [9] D. Karger et al., "Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the world wide web" in *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing*, 1997.
- [10] G. DeCandia et al., "Dynamo: Amazon's Highly Available Key-value Store" in *SOSP '07: Proceedings of the twenty-first ACM SIGOPS symposium on Operating systems principles*, 2007.

- [11] TiKV Project, "TiKV: Deep dive introduction" [Online]. Available: <https://tikv.org/deep-dive/introduction/>. [Accessed: 21-Dec-2025].
- [12] G. Fowler, L. C. Noll, and K. Vo, "The FNV Non-Cryptographic Hash Algorithm" [Online]. Available: <https://tools.ietf.org/html/draft-eastlake-fnv-17>. [Accessed: 21-Dec-2025].
- [13] Apache Cassandra, "Dataset Partitioning: Consistent Hashing" [Online]. Available: <https://cassandra.apache.org/doc/5.0/cassandra/architecture/dynamo.html#dataset-partitioning-consistent-hashing>. [Accessed: 21-Dec-2025].
- [14] L. Lamport, "The Part-Time Parliament" in *ACM Transactions on Computer Systems*, vol. 16, no. 2, May 1998.
- [15] L. Lamport, "Paxos Made Simple" in *ACM SIGACT News (Distributed Computing Column)*, vol. 32, no. 4, Dec. 2001.
- [16] Docker Inc., "Docker Documentation" [Online]. Available: <https://docs.docker.com/>. [Accessed: 19-Jan-2026].

Appendix A

Installation

This appendix provides description of where to find the project, how to set it up and how to run it for both testing and development purposes.

A.1 Project location

Project is located on author's GitHub repository at <https://github.com/DominikSzczepaniak/thesis-distributed-cache>.

A.2 Project Structure

The project is organized into a modular directory structure that separates different concerns and components. Below is a description of the main directories:

- **cmd/** - Contains the main entry points for executable applications:
 - **controller/** - Entry point for the Control Plane server
 - **datanode/** - Entry point for the Data Plane server
 - **raftcli/** - Command-line interface for interacting with the cache
- **pkg/** - Contains all reusable packages and core implementation logic:
 - **raft/** - Complete implementation of the Raft consensus algorithm, including leader election, log replication, persistence, and snapshotting
 - **controller/** - Control Plane logic, including topology management, rebalancing, and node health monitoring
 - **datanode/** - Data Plane logic, including data storage, replication, and shard migration
 - **client/** - Smart Client library with topology caching and query routing

- **api/** - HTTP API server implementation, including idempotency handling and retry logic
- **cache/** - Core in-memory key-value storage with concurrent access support
- **metadata/** - Cluster topology data structures and shard mapping logic
- **cachemodel/** - Shared data models and interfaces
- **tests/** - Testing infrastructure:
 - **benchmark/** - Performance benchmarking tools, including RPS measurement, scaling analysis, and comparison with other systems
 - **e2e/** - End-to-end integration tests
 - **fault_injection/** - Tests for failure scenarios, network partitions, and recovery mechanisms
- **deploy/** - Deployment configurations, including Dockerfiles for controllers and datanodes
- **scripts/** - Utility scripts for cluster management, including starting, stopping, and cleaning up the cluster
- **proto/** - Protocol Buffer definitions for gRPC communication (contains `raft.proto`)

The root directory also contains Docker Compose files for different deployment scenarios (`docker-compose.yml`, `docker-compose.node100.yml`, `docker-compose.node101.yml`) and shell scripts for local development (`run_local.sh`).

A.2.1 Prerequisites

System requires Docker to be installed with Docker Compose addition. The system is meant to be run inside a Docker container. If user wants to interact with system using author's CLI tool, then Go 1.24.1 or higher is required.

A.2.2 Setup

If user wants to test the system and does not need to scale it yet, it is better to set up the project on local computer as it's faster and more convenient.

The easiest way to start the cluster is using the provided script in the root directory of the project: `run_local.sh`. This script provides an interactive menu that automates the deployment and management of the system. It handles the following tasks:

- **Cluster Lifecycle:** It builds the necessary binaries and starts a base cluster consisting of three Controllers (forming a Raft quorum) and one initial data node.
- **Dynamic Scaling:** It allows the user to easily scale the cluster by adding additional data nodes (`datanode-2` through `datanode-5`) at runtime. This is particularly useful for observing the automatic shard rebalancing mechanism in action.
- **Fault Simulation:** It provides an option to stop specific data nodes, allowing the user to verify the system's resilience and its ability to promote replicas to primary status.
- **Interactive Testing:** It can launch the `raftcli` tool directly inside the Docker network. This allows for immediate testing of the cache with commands like `put` and `get`. Furthermore, the CLI's `load` command can be used to insert mock data (e.g., thousands of keys) to test the system's performance and data distribution.
- **Observability:** It includes a shortcut for viewing real-time logs of any specific service, which is essential for debugging and monitoring the Raft consensus process.

However, if user wants to do it himself, user should define a docker compose file with controllers for Control Plane. It is recommended to use at least 3 controllers, as only 1 will not guarantee any availability when it fails.

The docker compose example is given in the root directory as `docker-compose.yml`. It has defined:

- Controllers - the Dockerfile for building them is given in `deploy/Dockerfile`, however user can create any Dockerfile. It should use `cmd/controller` package to compile the binary. The Controller in Dockerfile needs a list of environment variables to be set:
 - `RAFT_ID`: The unique numeric identifier of the node (e.g., "0", "1", "2").
 - `TOTAL_NODES`: The total number of controllers in the cluster.
 - `RAFT_ADDRS`: A comma-separated list of addresses for all controllers (e.g., `controller-0:9000,controller-1:9000`).
 - `FILENAME`: The path where the Raft log and metadata will be persisted.
 - `SNAPSHOT_THRESHOLD`: The number of log entries that trigger a snapshot.
 - `API_ADDR`: The address on which the internal and external HTTP API will listen (e.g., ":8080").

- `API_IDEMPOTENCY_TTL`: The time-to-live for idempotency tokens in the cache.

However if user wants, he can further configure the system by defining:

- `RAFT_ARTIFICIAL_DELAY`: Simulates network or CPU lag for performance testing (e.g., "10ms").
- `RAFT_INITIAL_BACKOFF`: Initial delay for gRPC reconnection attempts.
- `RAFT_MAX_BACKOFF`: Maximum delay for reconnection attempts.
- `RAFT_BACKOFF_MULTIPLIER`: Factor by which the backoff increases after each failed attempt.
- `RAFT_MAX_RETRIES`: Maximum number of reconnection retries (−1 for infinite).
- `RAFT_CONN_TIMEOUT`: Timeout for establishing a gRPC connection.
- `RAFT_HEALTH_CHECK_INTERVAL`: Frequency of health checks for cluster nodes.
- `API_RETRY_MAX_ATTEMPTS`: Maximum retries for internal API requests.
- `API_RETRY_INITIAL_DELAY`: Initial delay before retrying an API request.
- `API_RETRY_MAX_DELAY`: Maximum delay between API retry attempts.

Controllers should also have exported ports and defined volumes for the data saves to be persistent.

- data nodes - the Dockerfile is given in `deploy/Dockerfile.datanode`. They use the `cmd/datanode` package. Required variables:
 - `CONTROLLER_URL`: The address of one of the controllers to perform registration and heartbeat (e.g., `http://controller-0:8080`).
 - `NODE_ID`: The unique identifier/address of the data node itself.
 - `LEASE_DURATION`: The time after which a node is considered dead if no heartbeat is received.

data nodes should also have exported ports and should be dependent on any of the controllers, so it can register and heartbeat.

Multi server setup

If user wants to use multiple servers, he should fetch the repository on each of them and run the same `docker-compose.yml` file. Note that, controller addresses have to be manually set by user in that file. It is recommended to start the containers around the same time on each of the computer, but its not required.

Data nodes can be added at any time, as long as they point to a correct controller IP address. The Control Plane is meant to handle the registration automatically.

Please note that this implementation is designed for demonstration and research purposes and is not intended for production environments. It lacks advanced features such as edge proxying or industrial-grade load balancing, which should be provided by specialized infrastructure in a commercial deployment. Furthermore, the setup requires robust network connectivity; all controllers and data nodes must be able to communicate bidirectionally. Users must ensure that the networks defined in the `docker-compose.yml` file are properly routed across physical hosts and that all necessary ports are correctly mapped and accessible.

A.3 Using the CLI

The CLI gives a simple interface to test the system. It supports cache operations of GET, PUT, DELETE. It has to be started with command `raftclicontroller-0:8080,controller-1:8080,controller-2:8080` - the user has to pass all controller IPs for it to work. Then it is just a simple interface that accepts `(COMMAND-TYPE) (KEY) (VALUE-optional)`, for example `put 1 10` or `get 1`.

A.4 Performance Benchmarking

The benchmarks discussed in Chapter 7 can be reproduced using the scripts in `tests/benchmark`. These allow for automated comparisons with other systems like Redis and Etcd.